

CALIBRATION PROGRAM FOR CAMERAS 2D/3D (ARTIFICIAL VISION)

Gustavo Magalhães Lopes



Departamento de Engenharia Eletrotécnica

Instituto Superior de Engenharia do Porto

29 de junho de 2025

Este relatório satisfaz, parcialmente, os requisitos que constam da Ficha de Unidade Curricular de Projeto/Estágio, do 3º ano, da Licenciatura em Engenharia Eletrotécnica e de Computadores

Candidato: Gustavo Magalhães Lopes, 1210668@isep.ipp.pt

Orientação científica: Lobinho Gomes, lobinho.gomes@gislatica.pt

Empresa: Gislatica - Projecto e Fabrico de Sistemas Mecânicos Lda

Orientador: Fernando J. S. Carvalho, fjc@isep.ipp.pt



Departamento de Engenharia Eletrotécnica

Instituto Superior de Engenharia do Porto

29 de junho de 2025

Agradecimentos

Gostaria de expressar o meu profundo agradecimento a todos os que, de alguma forma, contribuíram para a concretização deste projeto.

Em primeiro lugar, agradeço ao Engenheiro Lobinho Gomes, da empresa Gislótica, pela orientação técnica, disponibilidade e apoio ao longo do desenvolvimento do projeto. A sua experiência e sugestões foram fundamentais para a boa execução deste trabalho.

Agradeço também ao Professor Fernando Carvalho, pelo acompanhamento académico, pelo incentivo e pela motivação constante durante todas as fases deste projeto, bem como ideias para ultrapassar muitos dos obstáculos encontrados.

Um profundo agradecimento aos meus colegas e amigos no ISEP, bem como na Gislótica com quem mantive uma grande cooperação durante o tempo de trabalho, mas também com quem pude disfrutar das pausas, almoços e viagens de carro.

Por fim, um agradecimento muito especial à minha família e à minha namorada, pelo apoio incondicional e paciência ao longo de todo este percurso académico.

A todos, o meu mais sincero obrigado.

Abstract

This project was developed at Gislotica – Projecto e Fabrico de Sistemas Mecânicos Lda, with the main objective of implementing a wizard-style interface (assistant-style application) designed to support users in the calibration process of a camera mounted on a robotic arm. The ultimate goal, following calibration, is to determine the camera's position and reference points within its environment with the desired level of accuracy.

The interface implementation was designed to ensure continuous interaction with the user, providing clear instructions and feedback throughout the calibration process, thereby promoting an intuitive user experience. To achieve this, and in line with the company's guidelines, suitable implementation technologies were evaluated and selected. These include: C++ as the primary programming language; the Qt framework for developing a user-friendly graphical interface, owing to its wide array of interactive widgets and ease of event handling; the SQLite3 database for storing and retrieving relevant calibration-related information, ensuring both data persistence and accessibility; and the OpenCV library, integrated to support and streamline the camera calibration process. It is worth noting that OpenCV's comprehensive computer vision capabilities enabled accurate calibrations, ensuring precise determination of the camera's position as well as the locations of objects in its surroundings.

With the objective achieved, the resulting interface is modular and offers the following features: it facilitates camera calibration for users with little or no technical background; includes a flexible backend that accommodates varying calibration pattern dimensions; filters out invalid images; captures relevant frames – with or without zoom – from video streams; and presents a robust and maintainable code structure suitable for future adaptations.

Keywords

Camera Calibration, C++, OpenCV, Qt Library, SQLite3, Wizard

Resumo

Este projeto foi desenvolvido na empresa Gislotica - Projecto e Fabrico de Sistemas Mecânicos Lda e tem como objetivo principal a implementação de uma interface wizard (programa de estilo assistente), que permite auxiliar os seus utilizadores no processo de calibração de uma câmara instalada num braço robótico. Desse modo, após o processo de calibração, pretende-se determinar, com a precisão desejada, a posição da câmara e pontos de referência no ambiente onde se insere.

A implementação da interface considerou a interação contínua com o utilizador, no sentido de lhe fornecer instruções claras e feedbacks que permitissem que o processo de calibração fosse intuitivo. Para o efeito, em conformidade com as orientações da empresa, foram estudadas tecnologias de implementação adequadas, destacando-se: C++ como linguagem principal; bibliotecas Qt para o desenvolvimento visual de uma interface amigável, considerando a sua ampla gama de widgets interativos e a simplicidade na codificação das ações de cada widget; a base de dados SQLite3, para armazenar e recuperar informações relevantes necessárias para a calibração, garantindo a persistência e acessibilidade dos dados; bibliotecas OpenCV, incorporadas para facilitar o processo de calibração da câmara. De referir que as capacidades abrangentes de visão computacional do OpenCV permitiram a obtenção de calibrações precisas, garantindo que a câmara determinasse com rigor a sua posição e as posições dos objetos em seu redor.

Alcançado o objetivo, obteve-se uma interface modular, que: facilita a calibração da câmara a utilizadores sem experiência técnica; contém um backend que permite calibrações flexíveis à dimensão de padrões; filtra imagens inválidas; captura imagens de interesse, com ou sem ação de zoom, a partir de vídeo; apresenta uma estrutura robusta e de fácil manutenção em potenciais adaptações futuras.

Palavras-Chave

Biblioteca Qt, Calibração de Câmaras, C++, OpenCV, SQLite3, Wizard

Table of Contents

AGRADECIMENTOS	I
ABSTRACT	II
RESUMO.....	III
1. INTRODUCTION	14
1.1. Background	14
1.2. Objectives	15
1.3. Timeframe	16
1.4. Report's Structure	17
1.4.1. Introduction	17
1.4.2. Tools and Technologies	18
1.4.3. Software Development and Implementation	18
1.4.4. Conclusions	18
1.4.5. Annexes and References	18
1.5. Theoretical Background: Camera Calibration	19
1.5.1. Intrinsic Parameters	19
1.5.2. Extrinsic Parameters	20
1.5.3. Distortion Models	20
1.5.4. Calibration Process	21
2. TOOLS AND TECHNOLOGIES	22
2.1. C	22
2.2. Qt Libraries	22
2.3. SQLite3	23
2.4. OpenCV	24
2.5. State of the Art	25
3. SOFTWARE DEVELOPMENT AND IMPLEMENTATION	27
3.1. Idea and Adaptation	27
3.1.1. Low-level programs (First program)	28
3.1.2. Low-level programs (Camera search and display)	30
3.2. Main Program	32
3.2.1. General Architecture	32
3.2.2. Wizard Class Adaptation	33
3.2.3. Introduction Page	36
3.2.4. Camera Information Page	36
3.2.5. Choice Page	40

3.2.6. Calibration through Image Uploads	41
3.2.7. Calibration through Live Camera and Frame Extraction	46
3.2.8. Conclusion Page	52
4. RESULTS	53
4.1. Results Obtained	53
4.1.1. Normal Calibration (No Filters, Varied Images)	53
4.1.2. Calibration with Fewer Images	55
4.1.3. Calibration with Images that Have Reflection on the Chessboard	57
4.1.4. Calibration with Images with an Instagram Distortion Filter	58
4.1.5. Live Calibration	59
4.2. Calibration Imprecisions and Limitations	62
5. CONCLUSIONS	63
5.1. Future Perspectives and Possible Enhancements	64
REFERENCES	65
ANNEX A – Project’s Final Code (.h)	66
ANNEX B – Project’s Final Code (.cpp)	70

List of Figures

Figure 1 - Result of calibration with undistortion filter applied.....	15
Figure 2 - Drawn chessboard pattern, recognized by the program.....	16
Figure 3 – Chronogram of the work developed.	17
Figure 4 – Types of distortion	20
Figure 5 – Non-distorted image for each type of distortion	21
Figure 6 – ISO C++ Logo.	22
Figure 7 – Qt Logo [2].	23
Figure 8 – SQLite Logo [3].....	24
Figure 9 – OpenCV Logo.....	25
Figure 10 - Chessboard during a calibration application runtime.	26
Figure 11 - Aftermath with distortion removal.	26
Figure 12 – First Qt connection.....	28
Figure 13 – Writing in the Text Editor.....	29
Figure 14 – Result of the first low-level program.	29
Figure 15 - First layout of the camera search and display program.	30
Figure 16 - getCameras() function	31
Figure 17 - Final state of the camera search and display program.	31
Figure 18 – General architecture of the program	32
Figure 19 - Default order of the wizard pages.....	33
Figure 20 - Manual indexing of the wizard pages.....	33
Figure 21 – “Next” button simulation.	34
Figure 22 – Flowchart of Choice Page.....	35
Figure 23 – Results of Choice Page.	35
Figure 24 – Introduction Page Layout.....	36
Figure 25 – Camera Info Page Flowchart.	37
Figure 26 – Camera Info Page Layout.	38
Figure 27 - Upload Button’s File Explorer.	38
Figure 28 – File displayed as a result of the upload.	39
Figure 29 – Results of the query after the upload.	39
Figure 30 – Editing uploaded file’s text in Camera Info Page	40
Figure 31 – Choice Page Layout.	40
Figure 32 – Image Calibration Page Layout.	41

Figure 33 – QComboBox in Action for Selection of Images.	42
Figure 34 – Images Uploaded in the Database.	42
Figure 35 – Flowchart for the Download and Upload of Images.	43
Figure 36 – Example of a QWarning.	44
Figure 37 – Flowchart for the <i>calibrateImages()</i> function.	45
Figure 38 – Image Calibration Page Flowchart.	46
Figure 39 – Live Calibration Page Layout.	47
Figure 40 – Live Camera Calibration QComboBox’s Functions Flowchart.	47
Figure 41 - Live Camera Calibration Frame Extraction Related Functions Flowchart.	48
Figure 42 – Results of Clicking the Frame Save Button.	49
Figure 43 – Frame Saved in the Folder.	50
Figure 44 – Live Camera Calibration’s Calibration Related Functions.	50
Figure 45 – Ongoing Live Calibration.	51
Figure 46 – Undistorted Camera Opened.	52
Figure 47 – Conclusion Page Layout.	52
Figure 48 – Examples of images uploaded for a normal calibration.	54
Figure 49 – Example of an undistorted image for a normal calibration.	54
Figure 50 – Single image calibration (inclined chessboard)	55
Figure 51 - Single image calibration (chessboard with no inclination and centered)	56
Figure 52 - Single image calibration (chessboard with rotation, uncentered)	56
Figure 53 – Two image calibration with rotated boards.	57
Figure 54 – Results for a calibration with light reflection.	57
Figure 55 – Chessboards not found during the light reflection calibration	58
Figure 56 - Examples of images uploaded for a normal calibration	58
Figure 57 - Example of an undistorted image for a normal calibration	59
Figure 58 – Chessboard detection (live calibration results)	60
Figure 59 – Undistortion camera open	61
Figure 60 – Single frame calibration with live camera	61

List of Tables

Table 1 – Camera Matrix variables	19
Table 2 – Camera Matrix obtained from a normal and successful calibration.....	55
Table 3 – Distortion Coefficients obtained from a normal and successful calibration	55

Acronyms

ACID	-	Atomicity, Consistency, Isolation, Durability
CNNs	-	Convolutional Neural Networks
GCC	-	GNU Compiler Collection
GUI	-	Graphic User Interface
ICC	-	Intel C++ Compiler
IBM	-	International Business Machines Corporation
MATLAB	-	MATrix LABoratory
MOC	-	Meta-Object Compiler
MSVC	-	Microsoft Visual C++
R&D	-	Research and Development
SQL	-	Structured Query Language
UI	-	User Interface
YOLO	-	You Only Look Once

1. INTRODUCTION

1.1 Background

The project titled "Calibration Program for 2D/3D Cameras (Artificial Vision)" was proposed by the employing entity Gislotica - Projecto e Fabrico de Sistemas Mecânicos Lda and made available for applications through ISEP as part of the curricular unit PESTA. At Gislotica, it was decided that Engineer Lobinho Gomes, R&D specialist in Artificial Vision, would oversee and supervise the project.

The main objective of this project was to support the company by developing a software tool capable of calibrating cameras that would guide a user through planned steps displaying the results for later use. The original idea consisted of the calibration of a robot's camera and aimed to improve the precision of the robot's movements and its "perception" of the workstation environment in which it operates. While Gislotica already possessed a calibration program, it was limited in usability and tailored to a narrow set of parameters for a specific machine.

The new software developed as part of this project is designed to be more user-friendly and adaptable to a wider range of cameras, although this generalization may come at the expense of some calibration precision. Following the guidance of Engineer Lobinho, the software was developed in C++, primarily because the existing calibration tool at Gislotica was also written in this language. This compatibility allows for components of the newly developed program to be integrated into the existing system, and vice versa, if required.

The tools employed in the development of this project were also recommended by Engineer Lobinho, who specifically advised the use of Qt for the user interface. His recommendation was based on Qt's versatility and comprehensive functionality, which he anticipated would address any general or specific challenges that might appear during the project's development. The C++ Qt library offers a broad range of classes and functions that are essential for effectively translating the visual components (frontend) with the underlying

logic and computations (backend). Additionally, Qt's relatively simple and intuitive syntax contributed to a more time-efficient development process.

For the computer vision components of the project, the OpenCV library was chosen due to its extensive documentation and widespread use. Among its many resources, the camera calibration section of the official OpenCV documentation proved to be particularly valuable and frequently referenced throughout development. This comprehensive support facilitated the smooth integration of OpenCV into the project, especially following the completion of the user interface. Its robust functionality and ease of use made it a suitable and effective tool for implementing the vision-related features of the application. More information about the tools and technologies used can be found in section 2.1 of this document.

1.2 Objectives

The main objective of this project was to create an intuitive wizard for robotic camera calibration, focusing on two key operations: calculating the distortion coefficients and camera matrix, and using a filtering function based on these parameters to produce an undistorted output image, as illustrated from left to right in Figure 1.

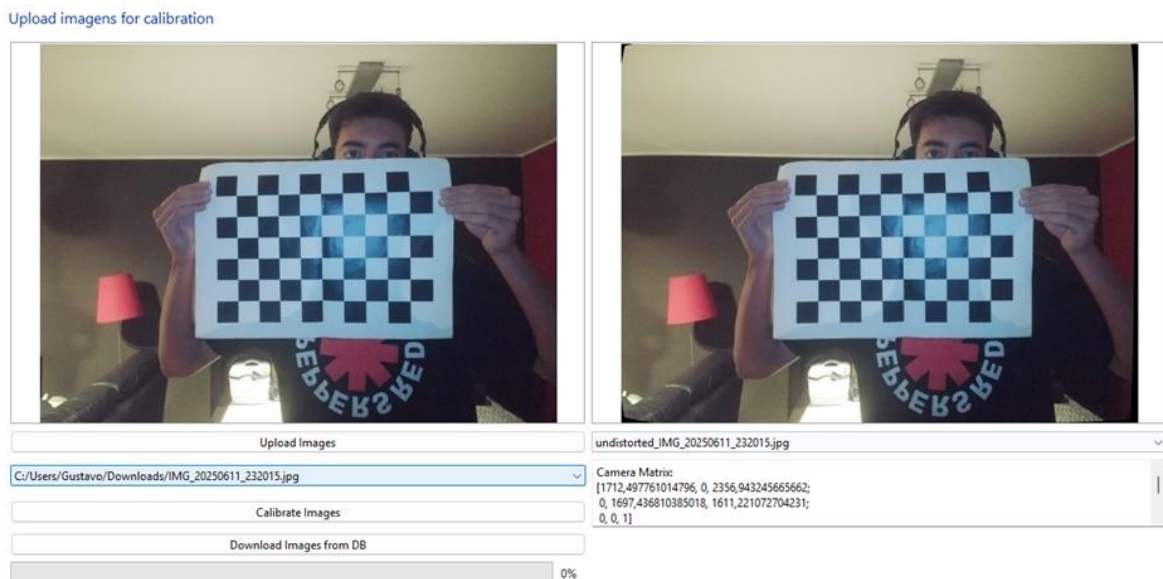


Figure 1 - Result of calibration with undistortion filter applied.

The calibration is based on the recognition of the chessboard as a pattern for measuring the distortion that needs to be corrected, as shown in Figure 2.

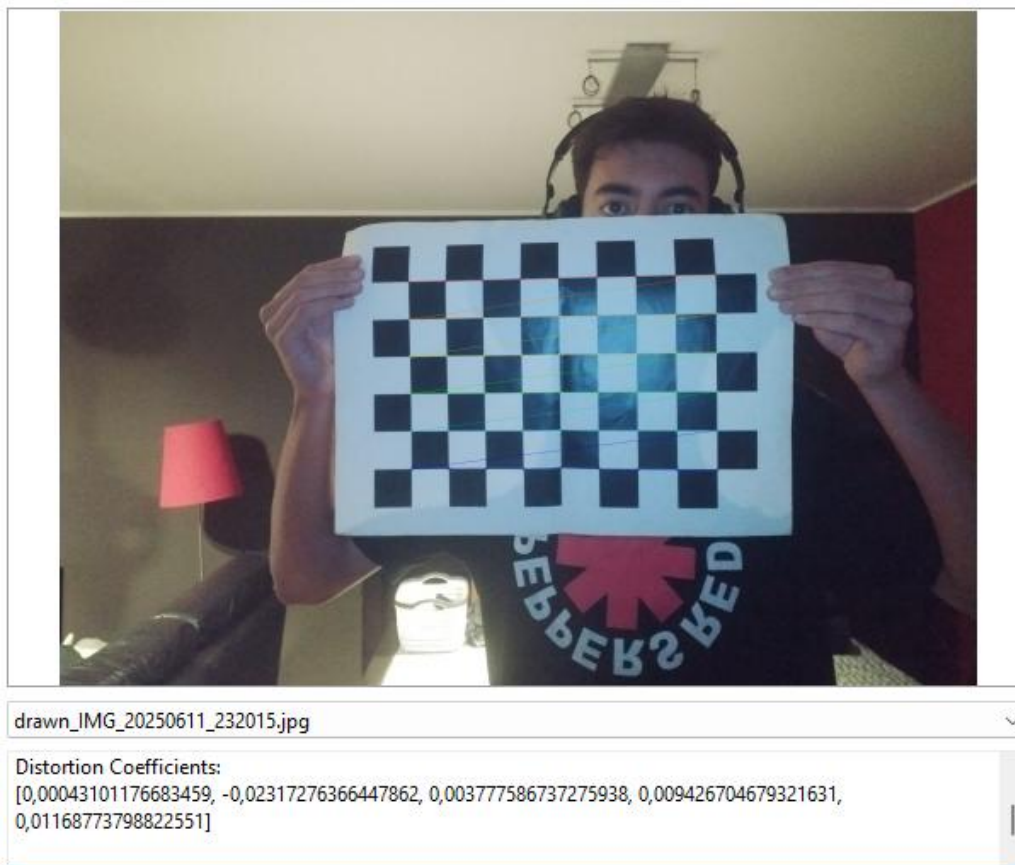


Figure 2 - Drawn chessboard pattern, recognized by the program.

The specific objectives for this project are:

- Create a user-friendly interface that allows everyone to use the program.
- Creating the code responsible for connecting the user interface (UI) with the logic and computation.
- Create a database that allows the user to store information.

1.3 Timeframe

The internship began in the previous year, 2024, and the project was developed in two main periods: the first between April 12 and June 12, 2024, and the second between March 15 and June 15, 2025. The work was structured into several phases, as outlined in the diagram presented in Figure 3, which provides a synthesis of the tasks carried out during these periods.

The initial phase involved integration into the company, becoming familiar with its internal processes, and meeting both the supervisors overseeing the project and colleagues who

would provide support throughout its development. Following this, a learning phase was dedicated to understanding the camera calibration process, the programming language (C++), and the tools involved. During this stage, several low-level programs were developed with the aim of internalizing key concepts; some of these foundational programs were later incorporated into the main application.

The final phase was the development of the project itself, which brought its own set of technical challenges.

The UI was developed first, serving as the foundation on which the process of calibration would be implemented. Following its completion, a low-level calibration program using OpenCV was integrated into the application and subsequently enhanced to improve functionality and usability.

Every aspect of the program went through a phase of enhancement, correcting errors and exploring ways to improve the user experience.

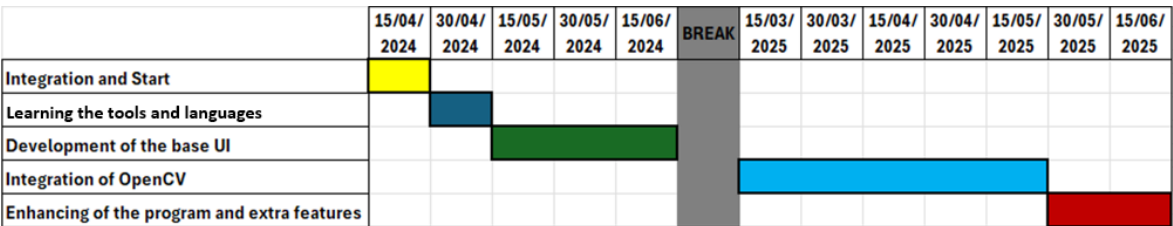


Figure 3 – Chronogram of the work developed.

1.4 Report’s Structure

This document is organized into clearly defined sections to facilitate a comprehensive understanding of the project, its development, and its outcomes. Any additional section can be found on the Table of Contents page (VII).

1.4.1 Introduction

Provides the project background, context, and objectives. It outlines the motivation for the development of a camera calibration tool and introduces the technologies and goals guiding

the implementation. It also includes a theoretical section that explains the calibration process in more detail.

1.4.2 Tools and Technologies

Details of the main programming languages, libraries, and frameworks used throughout the project. This includes C++, Qt, OpenCV, and SQLite3. Each tool is introduced with a description of its role and justification for its selection.

1.4.3 Software Development and Implementation

Explain the step-by-step development process of the application. It begins with early-stage low-level prototypes and progresses to the complete main program, structured as a Qt Wizard. Each subcomponent of the application is described, including the interface layout, backend logic, camera calibration processes (image-based and live), database integration, and error handling.

1.4.4 Results

This section presents the results obtained from the developed application. It evaluates the effectiveness of the camera calibration software by showcasing a variety of tests and experiments conducted under different conditions. These include calibrations using varied numbers of images, images with applied filters, lighting conditions, and the use of both static image uploads and live camera feeds.

1.4.5 Conclusions

This section reflects on the technical challenges encountered, the solutions implemented, future prospects and the project's possible impact on future calibration systems within industrial environments like Gislotica.

1.4.6 Annexes and References

Includes bibliographic references used during the development and documentation phases of the project as well as the final code of the program (header and cpp file).

1.5 Theoretical Background: Camera Calibration

Camera calibration is a fundamental process in computer vision and robotics that determines the intrinsic and extrinsic parameters of a camera. These parameters are essential for translating 2D image coordinates into real-world 3D coordinates and vice versa. Without calibration, measurements or spatial relationships derived from images can be inaccurate or distorted due to lens imperfections and geometric projection effects. [1]

1.5.1 Intrinsic Parameters

Intrinsic parameters define the internal characteristics of the camera, which influence how it maps the 3D world onto a 2D image. These include:

- **Focal length (f_x, f_y):** Describes the zoom level or magnification of the camera in pixel units.
- **Principal point (c_x, c_y):** The image center, ideally at the intersection of the optical axis with the image plane.
- **Skew coefficient:** Generally close to zero, this accounts for any non-rectangularity in the image sensor's pixel grid.
- **Distortion coefficients:** Account for lens distortion effects (radial and tangential), which warp the image, especially near the edges.

The parameters f_x, f_y, c_x and c_y can be extracted from the camera matrix. The camera matrix uses the following disposition [1][2][3][4]:

Table 1 – Camera Matrix variables

f_x	0	c_x
0	f_y	c_y
0	0	1

1.5.2 Extrinsic Parameters

Extrinsic parameters describe the position and orientation of the camera relative to the world coordinate system. These include:

- **Rotation matrix (R):** Describes how the camera is rotated relative to the scene.
- **Translation vector (T):** Describes the position of the camera in the scene.

Together, these allow the camera to locate itself within a known environment and are crucial for applications involving multiple viewpoints or robotic navigation.

1.5.3 Distortion Models

Real-world lenses introduce distortions that must be corrected during calibration. The two main types of distortions are:

- **Radial distortion:** Causes straight lines to appear curved. This is more pronounced toward the image edges (e.g., barrel or pincushion distortion). These types are exemplified in Figure 4.
- **Tangential distortion:** Occurs when the lens is not perfectly parallel to the image plane, causing asymmetry in image projection.

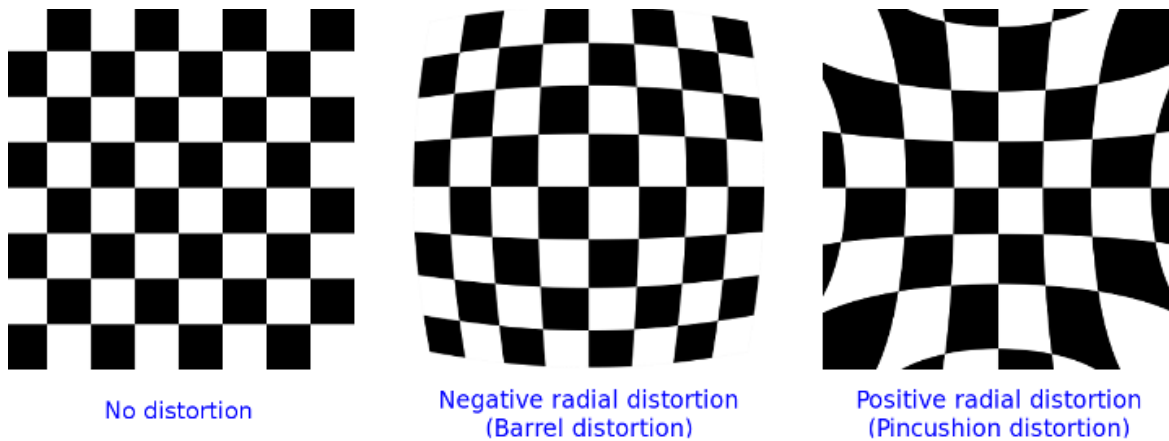


Figure 4 – Types of distortion [5].

The corresponding compensation observed when undistorting an image for these types of distortion can be seen in Figure 5.

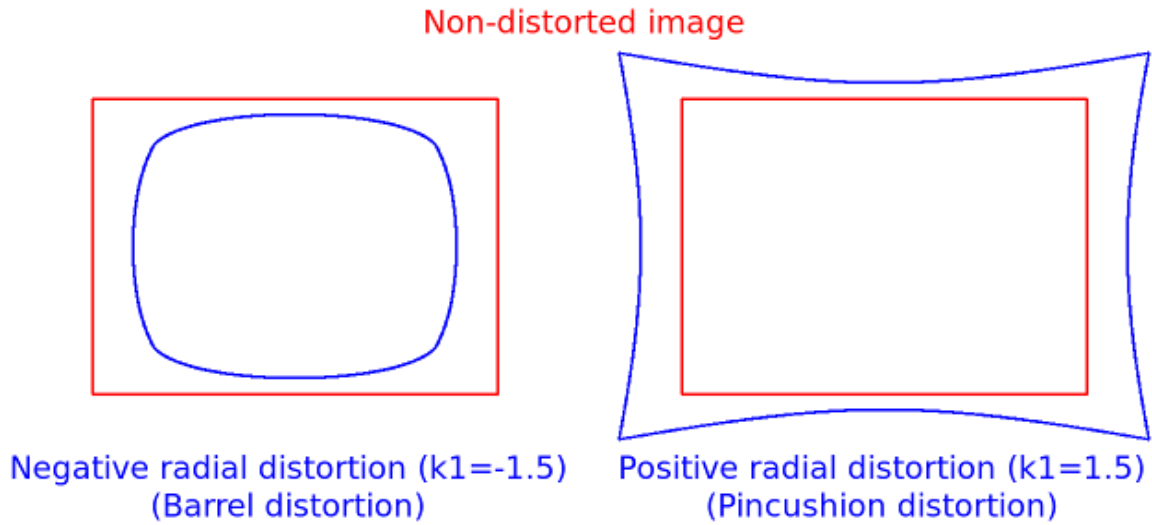


Figure 5 – Non-distorted image for each type of distortion [5].

OpenCV supports both distortion models and uses calibration data to remove these effects, producing undistorted images that more accurately represent the real world.

1.5.4 Calibration Process

The most common method for calibrating a camera involves using a known pattern (usually a chessboard or circle grid) [4] and capturing multiple images from different angles. The process typically involves the following steps:

1. **Detect feature points** on the calibration target in each image.
2. **Map these points** into their known 3D coordinates in the real world.
3. **Estimate the camera parameters** by minimizing the reprojection error, the difference between the observed image points and the projected 3D points using the estimated camera parameters. [3]
4. **Validate calibration** by applying the parameters to new images and verifying geometric accuracy.

2. TOOLS AND TECHNOLOGIES

This chapter covers all tools and technologies used in the making of this project, both hardware and software, and how the two parts connected.

2.1. C++

C++ is a versatile, cross-platform programming language widely used to create high-performance applications. Developed by Bjarne Stroustrup as an extension of the C language, C++ offers programmers significant control over system resources and memory management. It is one of the world's most popular programming languages, integral to many operating systems, graphical user interfaces, and embedded systems. Its object-oriented nature promotes a clear program structure and enables code reuse, reducing development costs. The language is highly portable, allowing applications to run on various platforms with minimal changes. Additionally, C++ is relatively easy to learn. Its similarities with languages like C, C#, and Java make it straightforward for programmers to switch between them, enhancing their versatility [6], Figure 6.

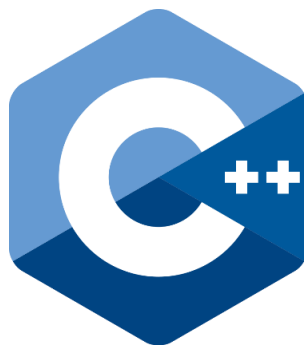


Figure 6 – ISO C++ Logo.

2.2. QT LIBRARIES

Qt is a cross-platform application development framework designed for creating applications on desktop, embedded, and mobile platforms. Supported platforms include Linux, OS X, Windows, Android, iOS, BlackBerry, and others, making Qt a versatile choice for developers targeting multiple operating systems. In the case of this project only the desktop application features have been used since the program created was meant for this platform. Qt is not a programming language on its own, it is a framework written in C++. It uses a preprocessor called the Meta-Object Compiler (MOC) to extend C++ with additional features such as signals and slots. Before the compilation step, the MOC parses source files written in Qt-extended C++ and generates standard-compliant C++ source files. This process ensures that both the framework and any applications or libraries using it can be compiled by standard C++ compilers like Clang, GCC, ICC, MinGW, and MSVC.

The development of Qt began in 1990 by Norwegian programmers Eirik Chambe-Eng and Haavard Nord. Their company, Trolltech, initially sold Qt licenses and provided support. Over the years, Trolltech underwent several acquisitions and is now known as The Qt Company, an owned subsidiary of Digia Plc.

One of the key features of Qt is its Widgets module, which allows developers to write graphical user interfaces (GUIs) directly in C++. Additionally, Qt includes an interactive graphical tool called Qt Designer, which can be helpful but was not used in the development of this project since all the widgets were created directly in the source code [7], Figure 7.



Figure 7 – Qt Logo [2].

2.3. SQLITE3

SQL (Structured Query Language) is a standard programming language used for managing and manipulating relational databases. It enables querying, updating, and managing data structures across various platforms and applications, ensuring robust data management.

SQLite is a widely used, high-reliability SQL database engine embedded in many applications and devices, including all mobile phones and most computers. It operates

without a separate server process, making it serverless and zero-configuration. SQLite reads and writes directly to disk files, with each database contained in a single file that is cross-platform compatible.

This SQLite file format is stable, backwards compatible, known for its compact size and efficiency. SQLite performs well even in low-memory environments and ensures that its transactions are ACID. It is extensively tested for reliability, achieving 100% branch test coverage.

SQLite is in the public domain, free for use, and supported by an international development team. Its versatility and ease of integration make it ideal for applications requiring a reliable, self-contained database solution, Figure 8.



Figure 8 – SQLite Logo [8].

2.4. OPENCV

OpenCV (Open-Source Computer Vision Library) is an open-source software library designed for computer vision and machine learning applications. This library is licensed under Apache 2, allowing businesses to freely use and modify the code.

OpenCV has over 2500 optimized algorithms that include both classic and state-of-the-art computer vision and machine learning functions. These algorithms allow the user to explore a list of diverse functionalities such as face detection and recognition, object identification, human action classification in videos, camera movement tracking, object tracking, 3D model extraction, 3D point cloud generation, image stitching, image similarity search, red-eye removal, eye movement tracking, scenery recognition, and augmented reality marker establishment.

It has a user community of over 47 thousand people and has been downloaded more than 18 million times. The library supports multiple programming languages, including C++, Python, Java, and MATLAB, and is compatible with Windows, Linux, Android, and Mac OS.

OpenCV is used across various industries and research fields, companies such as Google, Yahoo, Microsoft, Intel, IBM, Sony, Honda, and Toyota, as well as many startups and

research groups, that actively use OpenCV. OpenCV's deployed uses span the range from stitching street view images, surveillance video intrusion detection, mine equipment monitoring, robotic navigation, swimming pool accident detection, interactive art installations, runway debris checking and rapid face detection [9].



Figure 9 – OpenCV Logo.

2.5. STATE OF THE ART

As of today, camera calibration is a very common procedure, given the widespread use of cameras and their growing list of applications that benefit society. In the field of computer vision alone – as illustrated by OpenCV's deployed uses cases – we can point out intrusion detection, robotic navigation and face recognition [9], all of which are rapidly advancing. Some good examples of this would be the fact that nowadays a lot of smartphones can be unlocked through face recognition, and robotic navigation is being used to allow robotic waiters to serve customers in a lot of restaurants.

Another field on the rise that relies on computer vision and cameras is object recognition, more specifically, object recognition using AI-based algorithms. For object recognition, it's important to consider the type of camera used (because the learning algorithm and recognition rates vary when comparing a general and omnidirectional camera) [10]. For a robotic arm the most common type of camera is a general camera because it guarantees more precision in recognition. In some cases, omnidirectional cameras might be an option if the increased field of view is a priority.

One of the many uses of object recognition could be, for example, automatic video editing [11], saving hundreds of hours of manual editing. Imagine if you wanted to censor every person's face on a video you just captured (usually in public places) to conceal their identity, or censor a car's number plates, public advertisements, etc. Object recognition allows you to make this type of process automatic which in the long run could save a company or an independent videographer many hours of editing thus cutting costs of production.

The precision of object recognition algorithms is influenced by the calibration of a camera, and this implies that before running these algorithms it's important to have a well-calibrated camera. Therefore, it's necessary to have a program that can do this with efficiency and ease. Camera calibration can be achieved through various methods, some of the most simplest and efficient are planar calibration methods in which a chessboard with patterns and known measurements is stationary and the camera moves freely around it taking an extensive number of frames (this process can be performed either on a live camera or through images), as is shown in Figure 10.

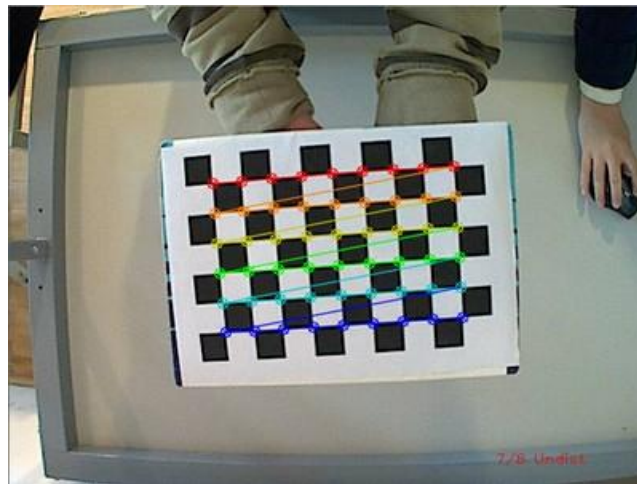


Figure 10 - Chessboard during a calibration application runtime.

Thus, the camera calibration process aims to cancelling distortions and find the relationship between camera's natural units (pixels) and the real world units (for example millimeters) [12], as shown in Figure 11.

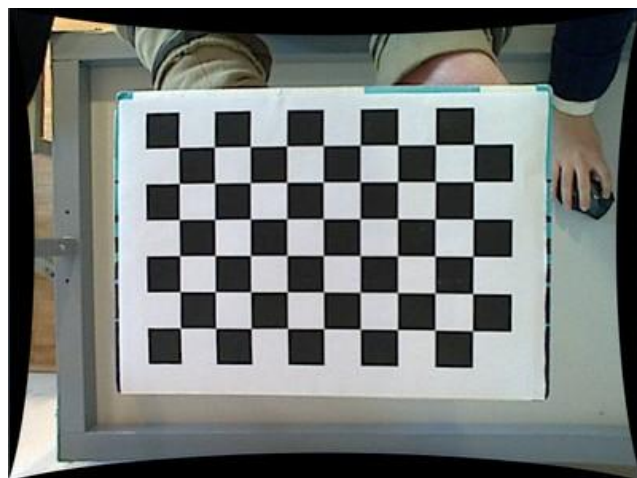


Figure 11 - Aftermath with distortion removal.

3. SOFTWARE DEVELOPMENT AND IMPLEMENTATION

3.1 Idea and Adaptation

The first step to creating this C++ program was to speak with Engineer Lobinho Gomes, the internship coordinator, so the idea behind this program could be better understood and all the requirements and details could be considered for the development of the project.

Eng. Lobinho recommended the use of the Qt libraries for the visual side of the program and suggested creating a wizard to make it easier for less experienced users to calibrate a camera when needed, as a wizard guides the user more clearly, helping them follow the necessary steps for calibration.

After collecting all this information, it was time to start developing the program, but as an unexperienced user of Qt it was necessary to learn the basics behind these libraries, so research on this subject was conducted and some low-level programs were developed before starting the main project. In this way it was easier to learn about the libraries compared to starting the main project without any knowledge on the subject. Later, some of these low-level programs were useful for the main project and some of the functions and visual aspects were implemented in the main project such as the camera selection and display, the slider, which was used to zoom the camera, the action functions for the buttons, etc.

For the calibration part, it was recommended to use OpenCV libraries and to try and apply a chessboard calibration method in my own personal camera with the objective of then using the same principles in a circle grid board on the robot's camera. Combining these two libraries, the program could run a simple and intuitive UI with Qt and allow a list of parameters and images to be fed into OpenCV's algorithms.

Knowing that data would be collected in this application, it was important to ensure it could be stored – both to recover it in case of errors and to allow the user to retrieve it later if needed. A database was considered a good and efficient solution. Following the

recommendation of Prof. Fernando Carvalho, and due to its many advantages in this type of application, SQLite3 was used.

3.1.1 Low-level programs (First program)

The first program that was created with the purpose of learning Qt was a simple Textbox that allowed the user to write something. After that the user could click on a Pushbutton to save the text that was previously written on the Textbox on a Label below it. In Figure 12, it's possible to see the implementation of a Qt connection created for the first program. This connection clears the text on the Textbox after the Pushbutton is clicked, so that after it is saved on the TextLabel the user can write something new.

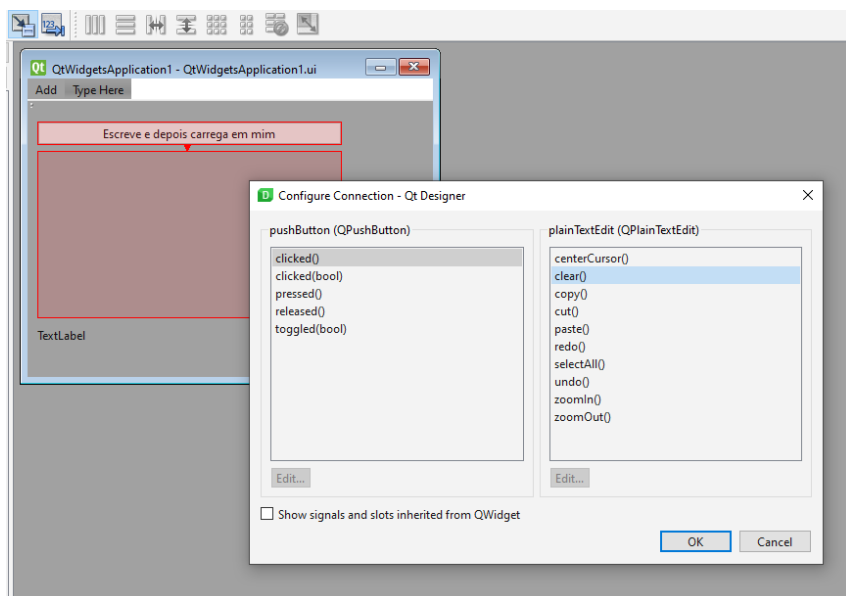


Figure 12 – First Qt connection.

Connections like the one shown in Figure 12 can be created in the Qt UI tool, also known as Qt Designer, or written directly in the source code. For these exercises, the connections were mainly made in Qt Designer, but for the main project, every single connection and UI aspect was coded directly into the source code. This allowed for easier maintenance and for every detail to be inspected directly as well.

The final state of the first program can be seen on the Figure 13 and Figure 14.

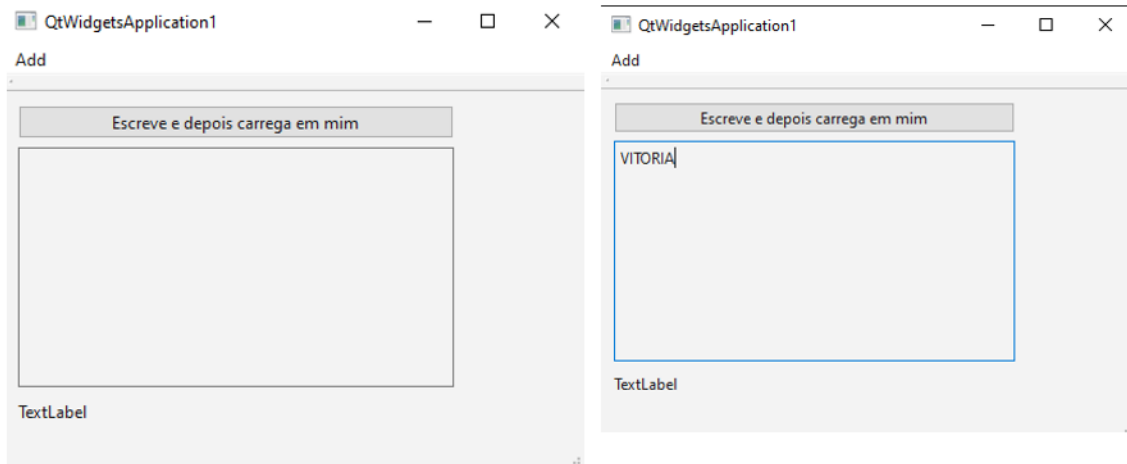


Figure 13 – Writing in the Text Editor.

In Figure 13, on the left, we can see how the first program appears when initialized, and on the right, what it looks after a user writes something on the Textbox. Finally, Figure 14 shows that the TextLabel successfully stores the inserted text.

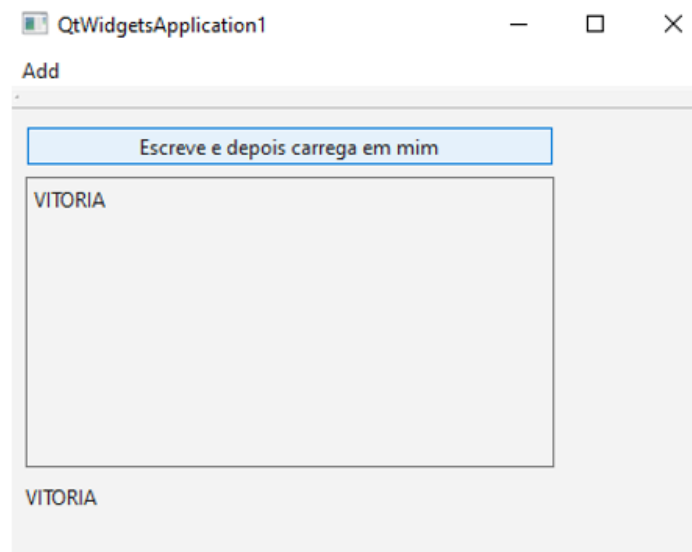


Figure 14 – Result of the first low-level program.

3.1.2 Low-level programs (Camera search and display)

The next program that was developed was capable of searching for multiple cameras connected to the user's device and listing them in a ComboBox, which was then used to select what camera to be displayed. It is possible using Qt because it has a set of libraries

called “Multimedia”. This set of libraries offers a variety of widgets and functions that let the developer manipulate cameras and other local devices.

In the Figure 15, we can see the first layout developed in QtDesigner, which consists of a ComboBox (used to select which device is being displayed) and a MediaCaptureSession (used to display the selected device).

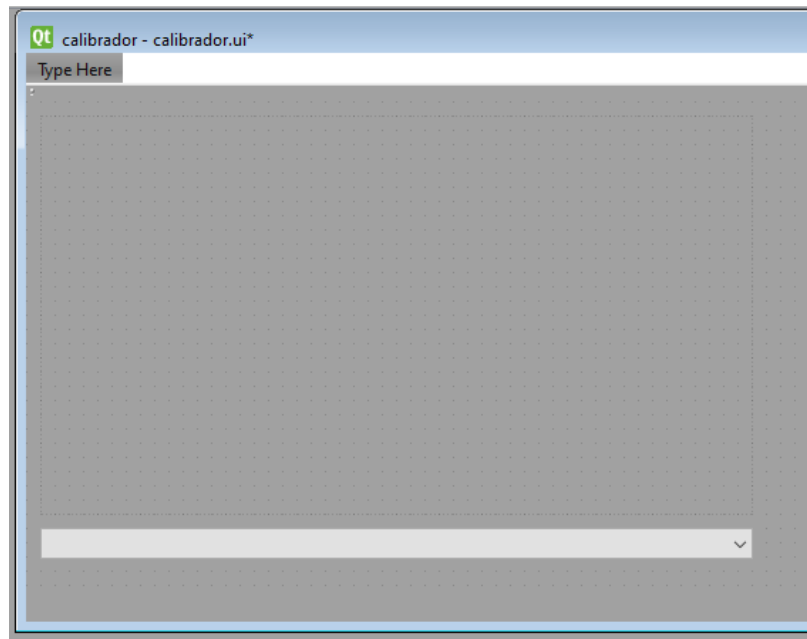


Figure 15 - First layout of the camera search and display program.

After that it was time to connect these two Widgets so that they could exchange information, this was achieved by the following code (see Figure 16), this section of the code gets all the information it needs (in this case, camera devices) from the computer in which the program was initialized, after that and after putting everything on a list which was then used by the ComboBox which waits for the user to select one of the cameras, when this happens the MediaCaptureSession displays the camera that was selected.


```

void CameraPage::getCameras()
{
    combocamaras->addItem("<None>");
    //coloca na lista "cameras" todos os dispositivos a captar video
    cameras = QMediaDevices::videoInputs();
    //adiciona à comboBox essa lista
    for (const QCameraDevice& camera : cameras)
    {
        combocamaras->addItem(camera.description());
    }
}

```

Figure 16 - getCameras() function.

For learning purposes (manipulation of display's aspect ratio, manipulation of sliders and other variable and interactive widgets) a zoom function was added to the program. This zoom function acted on a frame (that could be extracted by the user by clicking a PushButton) and let the user change the zoom factor using an interactive slider that varied from 0% zoom (zoom out) to 200% zoom (zoom in).

The final state of the program can be seen in Figure 17 and shows both the camera selection and display as well as the zoom being used in an extracted frame (it's important to notice that after being extracted the frame is totally independent of the camera being displayed in real-time).

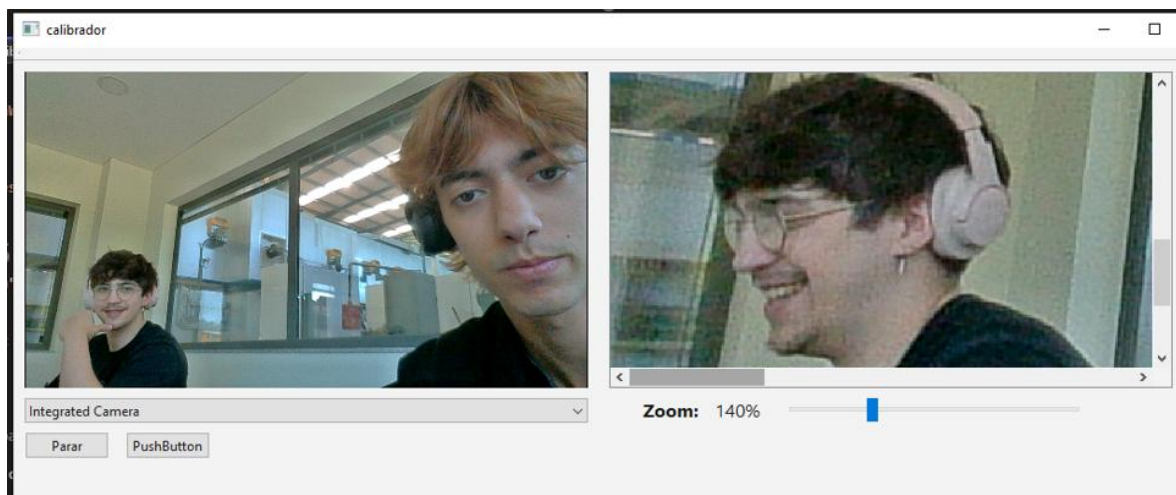


Figure 17 - Final state of the camera search and display program.

3.2 Main Program

This sub-section focuses on the main program and how it was achieved, how every process works based on user interaction and what are the possible outcomes of said actions.

3.2.1 General architecture

As stated previously, this program guides the user through the process of calibrating a camera. The general architecture of interface is shown on the Figure 18.

An introduction is made, followed by a file upload page in which the user uploads relevant information about the camera being used. Then, a “choice page” is presented, providing the user with the ability to choose between live calibration or image calibration (achieved by uploading different photos of the calibration target). When either calibration process is complete, the user is then presented with the conclusion page.

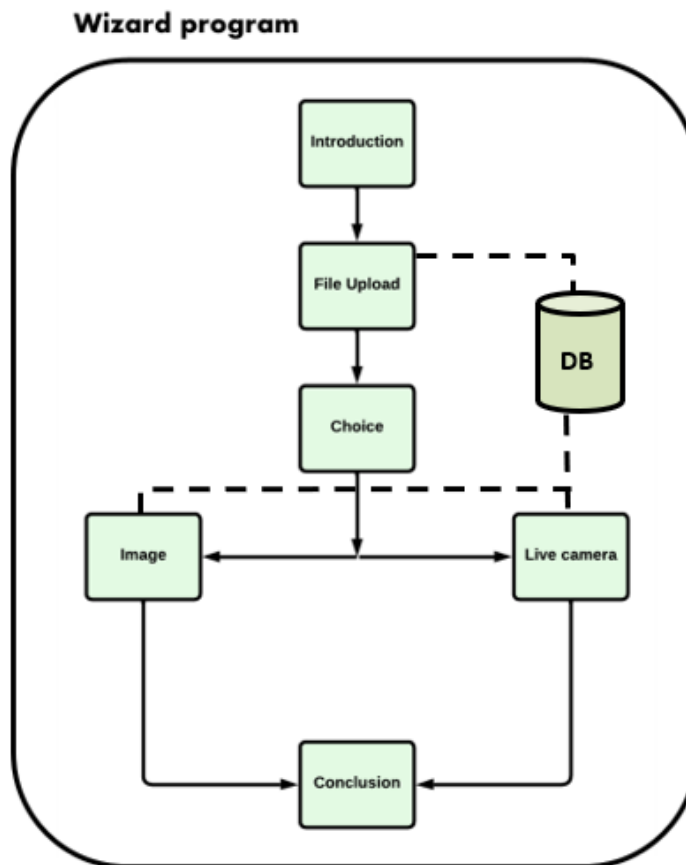


Figure 18 – General architecture of the program.

3.2.2 Wizard class adaptation

During the development phase of this project and halfway through implementation, it was concluded that the best approach would be to use the *QWizard* class for this program. The initial idea was to try to have a single main page where the entire calibration process could be carried out. The adaptation was not simple, as the *QWizard* class is not very well documented.

This class offers the option to index pages, so that when a user selects the type of calibration they desire on the choice page, the program selects the correct and intended page to display. If the pages are not indexed, the program follows the order in which the pages were added into the program, as seen in Figure 19.

```
~ClassWizard::ClassWizard(QWidget* parent)
: QWizard(parent)
{
    addPage(new IntroPage);
    addPage(new CameraInfoPage);
    addPage(new ChoicePage);
    addPage(new ImagesPage);
    addPage(new CameraPage);
    addPage(new ConclusionPage);

    setWindowTitle("Calibração de camaras");
}
```

Figure 19 - Default order of the wizard pages.

When manually indexing the pages, it's needed to change the "Id" of said pages in the header file, as seen in Figure 20.

```
public:
    ClassWizard(QWidget* parent = nullptr);

    static const int IntroPageId = 0;
    static const int CameraInfoPageId = 1;
    static const int ChoicePageId = 2;
    static const int ImagesPageId = 3;
    static const int CameraPageId = 4;
    static const int ConclusionPageId = 5;
```

Figure 20 - Manual indexing of the wizard pages.

Each page skips to the next page Id when the user clicks the "Next" *Pushbutton* located at the bottom of the current page, as seen in Figure 21.

On the choice page, where it is possible to select a different type of calibration, it was necessary to implement a condition when clicking on the “Next” *PushButton* to ensure the correct next Id is selected. This was done with a simple IF condition where the *nextID()* function returns the correct ID by analyzing which *RadioButton* is selected, as seen in Figure 21, Figure 22, and Figure 23.

The figure consists of two screenshots of a web application interface for camera calibration. The top screenshot shows the 'Camera Calibration' page with a sub-header 'Camera Information'. It contains a text area for the 'Camera Matrix' with the value `[2425,538163364997, 0, 2474,634026763856; 0, 2396,330441897921, 1618,966846627024; 0, 0, 1]`, an 'Upload file' button, and three input fields: 'Board Height (rows)' with value 6, 'Board Width (columns)' with value 9, and 'Square Size (in cm)' with value 3.5. At the bottom right are 'Next' and 'Cancel' buttons. The bottom screenshot shows the 'Camera Calibration' page with a sub-header 'Wizard for Camera Calibration'. It contains the text 'Choose between live calibration and image calibration:' and two radio buttons: 'Image Calibration' (selected) and 'Live Calibration'. At the bottom right are 'Next' and 'Cancel' buttons. A large red arrow points from the 'Next' button in the top screenshot to the 'Next' button in the bottom screenshot.

Figure 21 – “Next” button simulation.

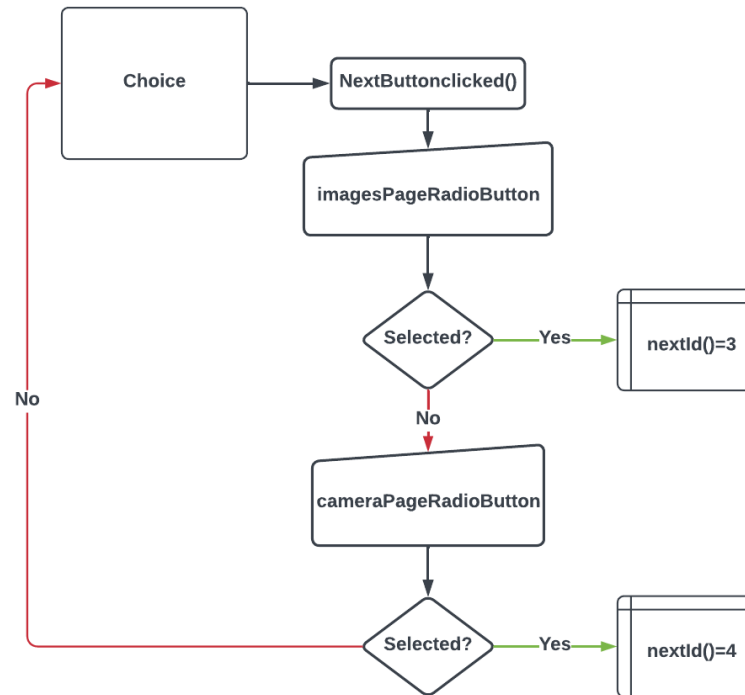


Figure 22 – Flowchart of Choice Page.

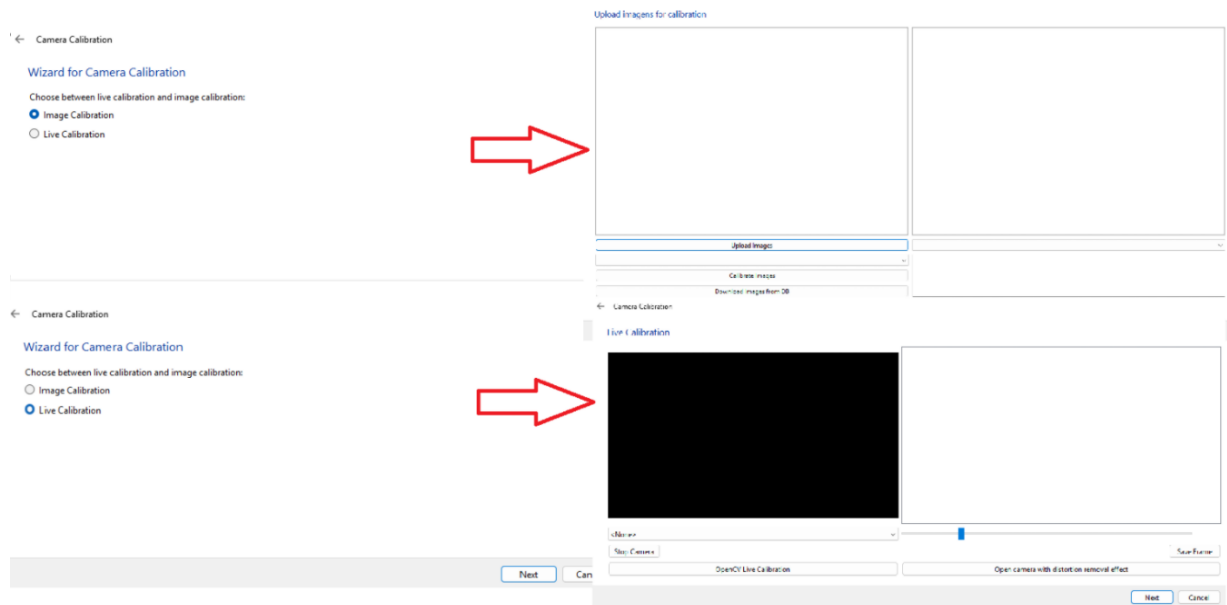


Figure 23 – Results of Choice Page.

3.2.3 Introduction page

The introduction page consists of a simple *QWizardPage* with a brief description of the calibration process and possible calibration methods that can be selected and used by the user. The page consists of a title, a Label with the program's description and the usual Next and Cancel buttons, as seen in Figure 24.

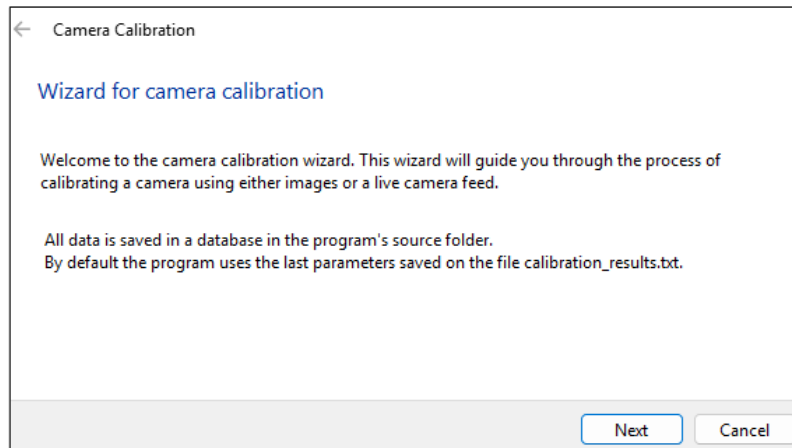


Figure 24 – Introduction Page Layout.

3.2.4 Camera Information Page

The succeeding page's intention is to let the user upload a file containing the camera's known information that's relevant to the calibration process. This file's type needs to be a Text File with the extension .txt and the information is then displayed so the user can check for possible flaws in the contents. All the contents must be in a certain format, and if they're not, the user is then advised to upload a file in the correct format and is provided with an example, Figure 25.

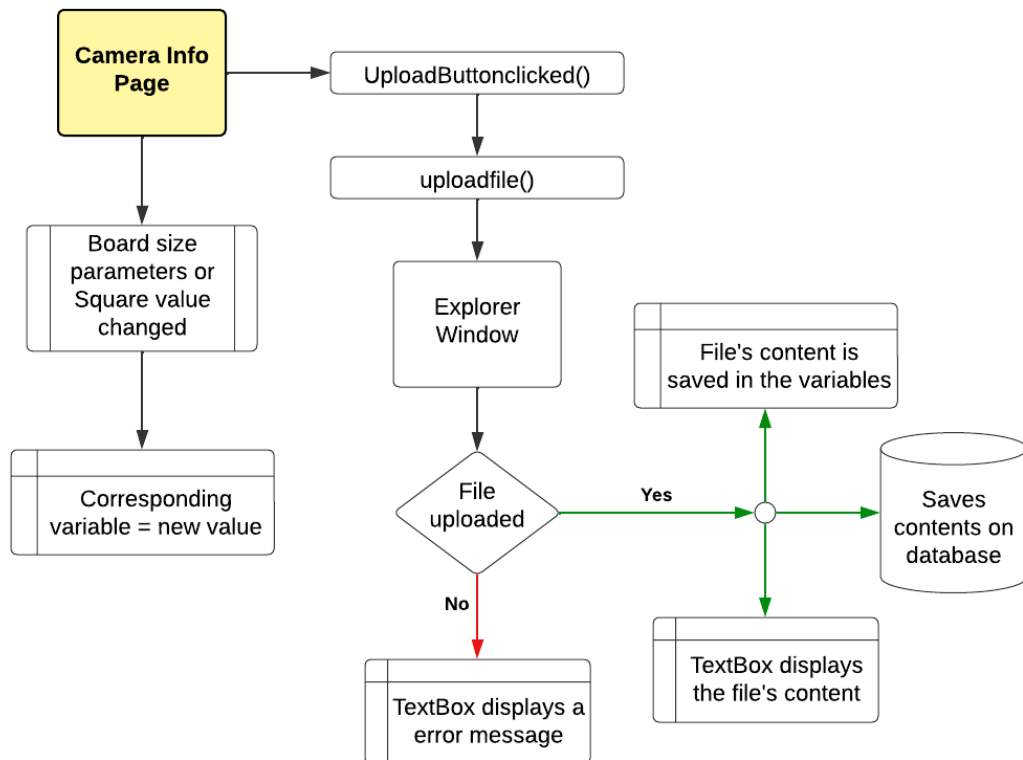


Figure 25 – Camera Info Page Flowchart.

It's also possible to select the number of columns and rows as well as the square size of the chessboard used, providing more accessibility to users with different chessboards.

The file's content is then saved in a local database on a table called *camerainfo* along with the upload date and the file's name, so that the information can be queried if needed by date or name. This feature is made possible by SQLite3 and its libraries. To manipulate this database three functions were created, one to initialize the database, one to upload the file and finally one to close the database.

When the page first initializes, the user is met with a title, a Label containing a brief description of what is intended to be uploaded, an empty *PlainTextBox* and a *PushButton* which provides the option to open a file explorer to upload a text file. It's possible to manually insert the camera information into the *PlainTextBox*, this feature is meant only for maintenance of the uploaded file's content since the manually inserted content is not saved in the database, Figure 26.

Figure 26 – Camera Info Page Layout.

If the upload button is clicked, the *uploadfile()* function is called which then opens an explorer window when the *filePath* variable is initialized, Figure 27.

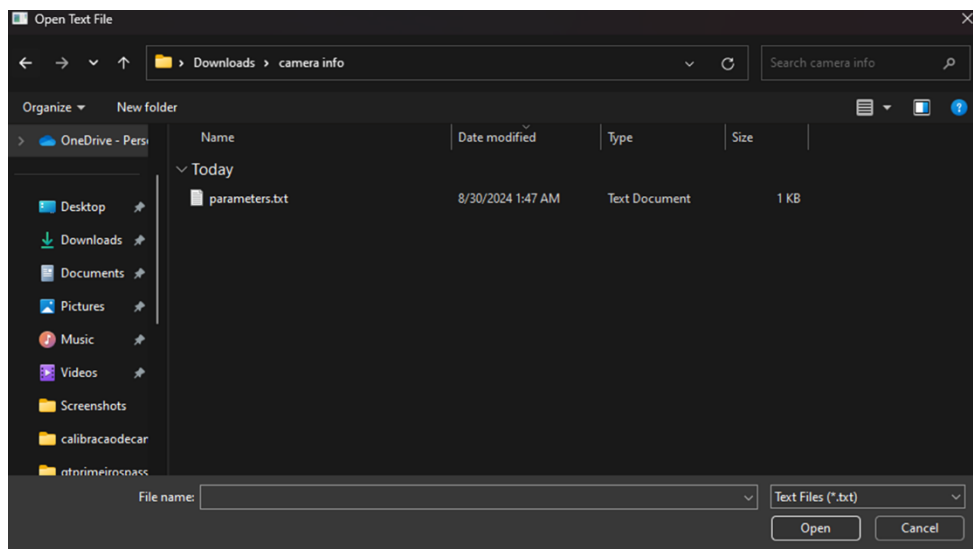


Figure 27 - Upload Button's File Explorer.

This window lets the user search and selects a text file with extension .txt, an error message is displayed if this upload process is either cancelled or fails. If the upload process is completed, the file's contents, name and date of upload are automatically saved in the local database, and the contents of this text file are displayed in the *PlainTextBox*, Figure 28.

Camera Information

To begin, upload the file with the camera information that will be relevant for the calibration and select the number of rows, columns and square size. This program will only consider these parameters valid for the Image Calibration Page, and will use the inputed parameters only once. If no information is inputed, the program will continue with default values and create new parameters for calibration:

Camera Matrix:
 [2425,538163364997, 0, 2474,634026763856;
 0, 2396,330441897921, 1618,966846627024;
 0, 0, 1]

Board Height (rows):

Board Width (columns):

Square Size (in cm):

Figure 28 – File displayed as a result of the upload.

Figure 29 presents the results of a query made on the local database uploadsdatabase.db, specifically the camerainfo table. This figure shows that the file information, along with its contents, was successfully saved.

```

26|2025-06-14|calibration_results.txt|Camera Matrix:
[2425,538163364997, 0, 2474,634026763856;
0, 2396,330441897921, 1618,966846627024;
0, 0, 1]
Distortion Coefficients:
[0,04001365133817153, -0,1155204594430027, 0,006206887292136207, 0,01640144146822343, 0,08781848984473301]
  
```

Figure 29 – Results of the query after the upload.

On Figure 30, we can see that it's possible for the user to modify the parameters or any information in the *PlainTextBox*, , allowing incorrect data to be corrected so that the program can function properly for the specific camera.

← Camera Calibration

Camera Information

To begin, upload the file with the camera information that will be relevant for the calibration and select the number of rows, columns and square size. This program will only consider these parameters valid for the Image Calibration Page, and will use the inputed parameters only once. If no information is inputed, the program will continue with default values and create new parameters for calibration:

Camera Matrix:

```
[2425,538163364997, 0, 2474,634026763856;
0, 2396,330441897921, 1618,966846627024;
0, 0, 1]
```

Board Height (rows):

Board Width (columns):

Square Size (in cm):

Figure 30 – Editing uploaded file’s text in Camera Info Page.

3.2.5 Choice page

The next page features the two possible options for calibration, the user is met with a title, a Label asking them what type of calibration they’d like to choose and two radio buttons with Labels attached so that they know which one to select, Figure 31.

← Camera Calibration

Wizard for Camera Calibration

Choose between live calibration and image calibration:

☐ Image Calibration

☐ Live Calibration

Figure 31 – Choice Page Layout.

After selecting a radio button and clicking the Next *PushButton*, the program identifies which page Id corresponds to which calibration option and redirects the user to the intended page. This process is made by a simple IF condition and if the user does not select any option,

the program stays on the same “Choice Page” waiting for a selection. The logic for this page can be seen in Figure 22.

3.2.6 Calibration through image uploads

This section features one of the user’s possible options for calibration, calibration through pre-taken images and references. This page consists of a title, two labels with the purpose of displaying different images, an upload button and a download button so that the user can upload images previously taken with the camera or previously used images from the database and *QComboBoxes* that allow for the selection of which image is being displayed in the corresponding Label. There is also a button (*CalibrationButton*) which after uploading images from the File Explorer or the database, allows the user to make a calibration of said images and displays the calibration results in the corresponding label and the parameters extracted in the Text Editor.

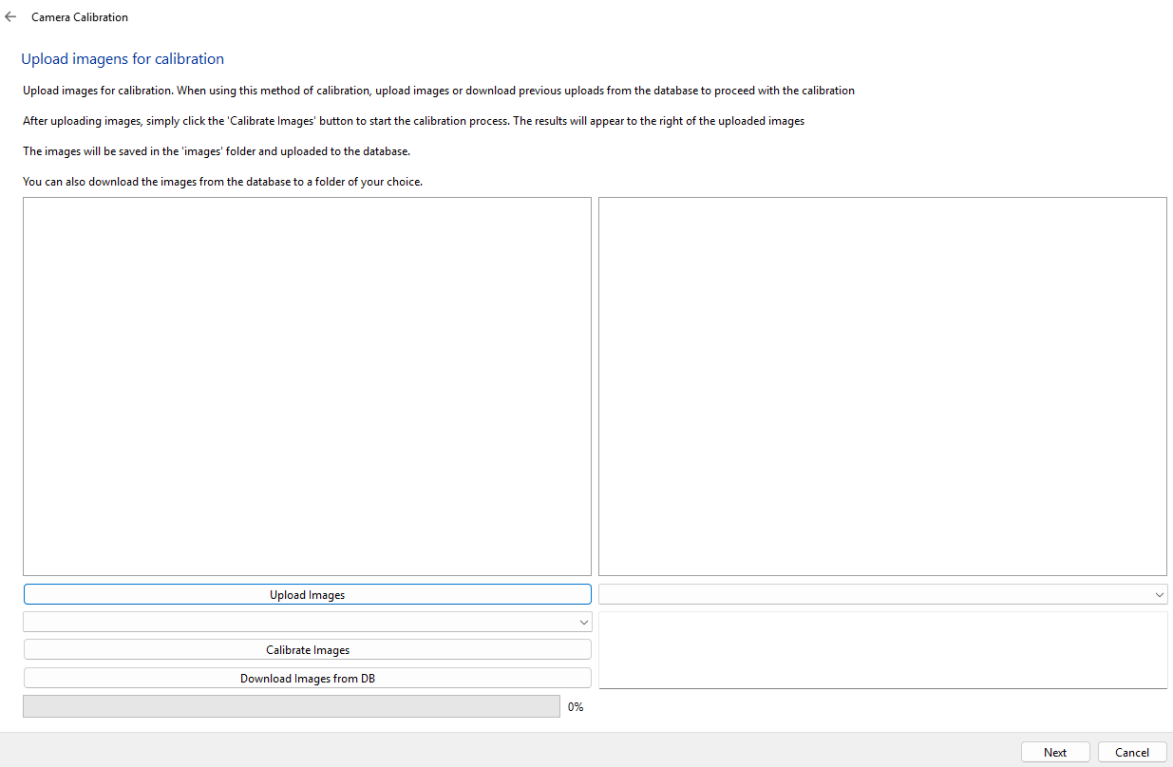


Figure 32 – Image Calibration Page Layout.

This method of calibration is intuitive and has three main *PushButtons* that are briefly explained in the subtitles of the page as seen in the layout of Figure 32. When the user clicks the “Upload Images” button, the `uploadImage()` function is triggered. This opens a file

explorer like the one used on the Camera Information page allowing the user to select the desired images for camera parameter extraction. Once selected, the images are uploaded to the database and displayed on the page, allowing the user to check for any unwanted files, Figure 33, and Figure 34.

It is important to note that the images are stored in binary (blob) format in the database. However, this format presents no issues when retrieving the images for further calibrations.

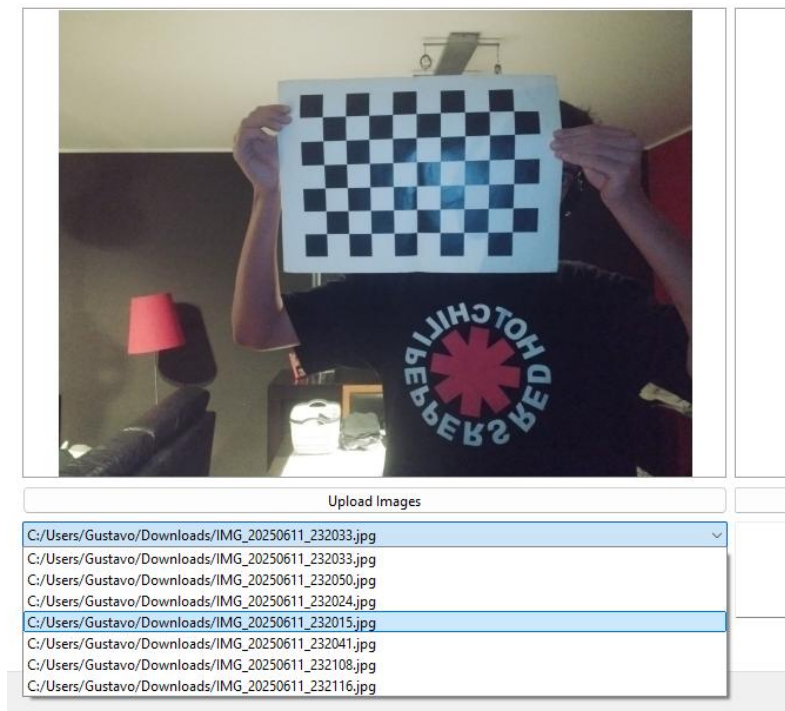


Figure 33 – QComboBox in Action for Selection of Images.

```
sqlite> SELECT * FROM imagesforcal;
373|2025-06-15|IMG_20250611_232033.jpg|♦♦♦♦
374|2025-06-15|IMG_20250611_232050.jpg|♦♦♦♦
375|2025-06-15|IMG_20250611_232024.jpg|♦♦♦♦
376|2025-06-15|IMG_20250611_232015.jpg|♦♦♦♦
377|2025-06-15|IMG_20250611_232041.jpg|♦♦♦♦
378|2025-06-15|IMG_20250611_232108.jpg|♦♦♦♦
379|2025-06-15|IMG_20250611_232116.jpg|♦♦♦♦
```

Figure 34 – Images Uploaded in the Database.

Before calibrating, if the user wishes to use images from the database they simply have to click on the “Download Images from DB” button. This *PushButton* calls a function that queries the table “*imagesforcal*” for the content and name of the file, if found, all the entries are then downloaded to a folder in the source directory allowing the user to upload these

images to the program with the “Upload Images” button, following the flowchart show in Figure 35.

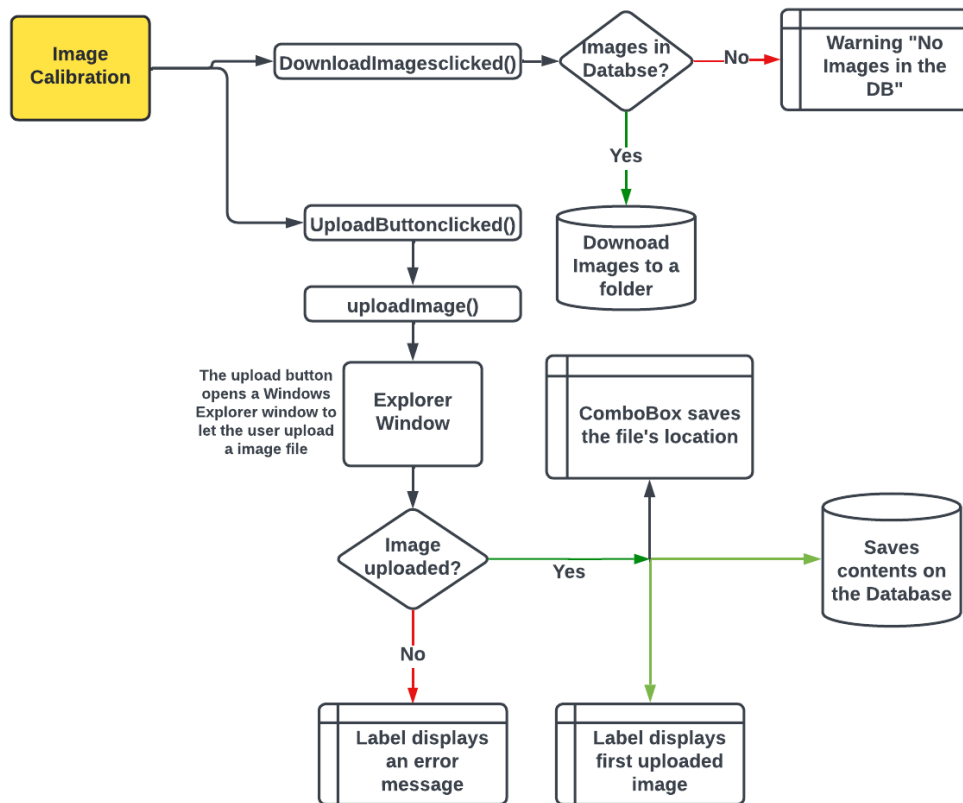


Figure 35 – Flowchart for the Download and Upload of Images.

After this the “Calibrate Images” button is ready to be used. The button triggers a function with the same name that takes no variables since the uploaded images are already on a known path in the source directory. This function *calibrateImage()* needs validation of a flag to work that is active only when the user has uploaded files into the program. It is a type of validation practice, used in almost all error control of the program, allowing it to continue running normally and not crash if the user ends up making a mistake or carrying out an action without the correct information being inputted. Thanks to this, the program guides the user by giving instructions on how to proceed, Figure 36.

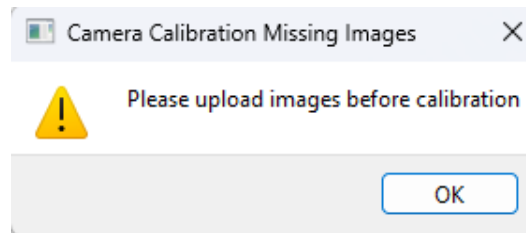


Figure 36 – Example of a QWarning.

If the flag is active, the function analyzes the board width, height and square size and creates arrays that will later be used to find the chessboard corner in the images. After which, it iterates through the directory “images” in the source folder and gets all the paths to the images that are going to be used for calibration (if this directory “images” is empty the program returns a warning message). Then using the function *findChessboardCorners()* available in OpenCV, it tries to filter the images that are valid for calibration (if no images are valid, it returns, with a warning message). After collecting all the images valid for calibration and their paths, the function *calibrateCamera()* from the OpenCV library is called, providing the distortion coefficients and the camera matrix. This information is stored in a file in the source folder for later use, if necessary. Finally, the program runs through the images that were considered valid for calibration, converts them to a matrix and uses the function *undistort()* of OpenCV’s library and stores the results in a folder.

After this entire process, the results (drawn images, undistorted images and camera’s parameters) are uploaded to the program’s corresponding widgets. Figure 35 shows a detailed flowchart of this function.

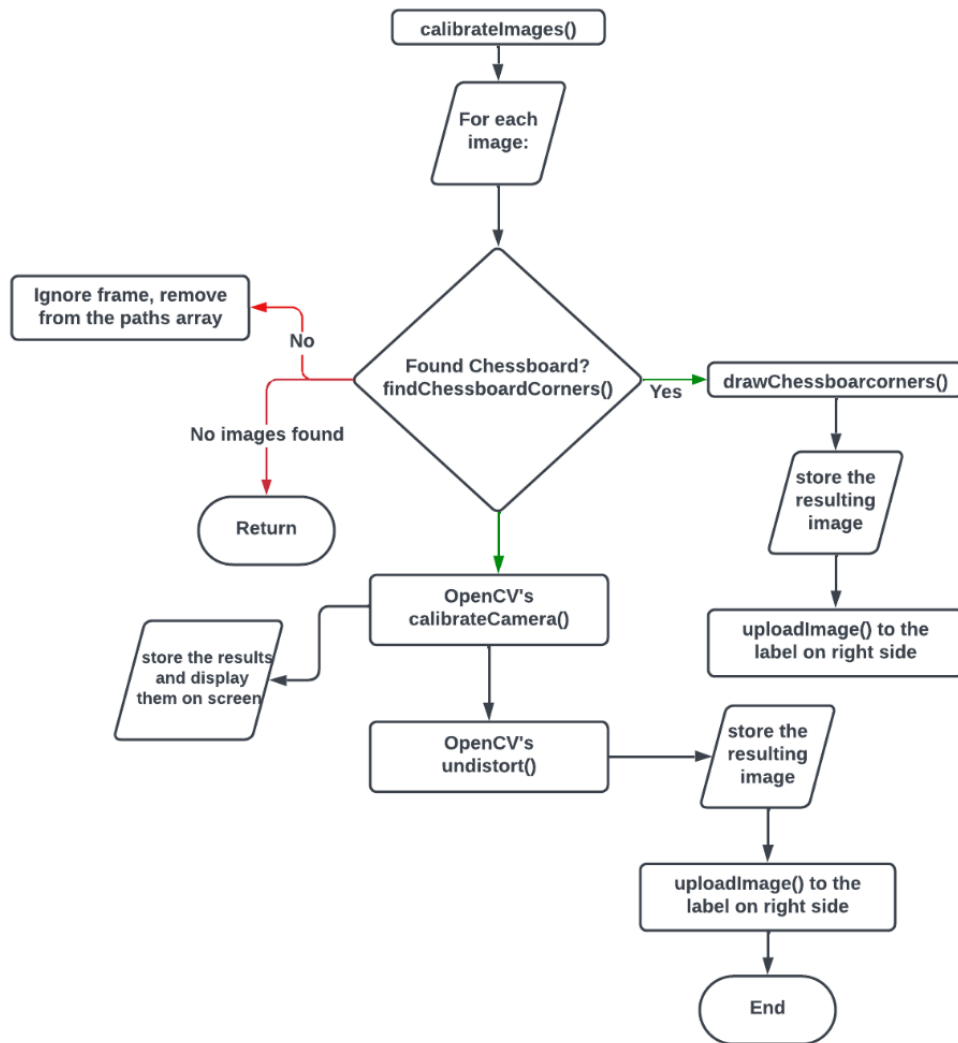


Figure 37 – Flowchart for the *calibrateImages()* function.

The calibration results are then displayed. The drawn images with the detected chessboard corners, as well as an image with the undistortion filter applied, are saved in the source folder. The parameters extracted from the image set are also saved and displayed in *QTextEdit*.

In the Figure 38 we can see the complete flowchart for this page.

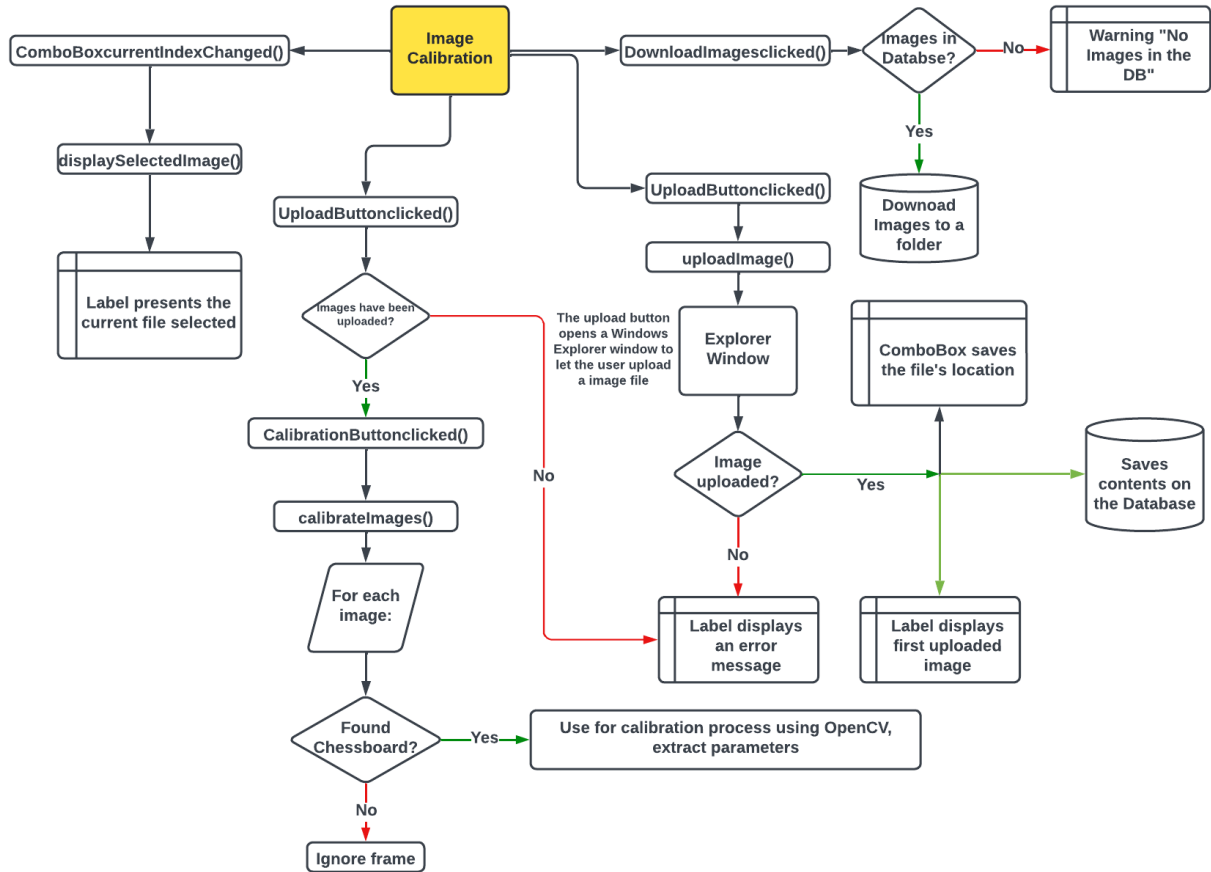


Figure 38 – Image Calibration Page Flowchart.

3.2.7 Calibration through live camera and frame extraction

The second calibration option takes inspiration from [3.1.2 Low-level programs \(camera search and display\)](#) and has all the features of that first program. Instead of using pre-taken images to calibrate the camera, the user can capture images in real time. This is made possible through a local camera search among the devices connected to the operating computer.

The page for this calibration option consists of a title, a *QVideoWidget* that displays the camera selected using a *QMediaCaptureSession* as an intermediary between this widget and the camera, a *QComboBox* showing all the cameras that were found in the operating computer, a button used to take frames from the camera being used, an image label to display these frames, and a slider used to zoom in the extracted frames, as seen in Figure 39.

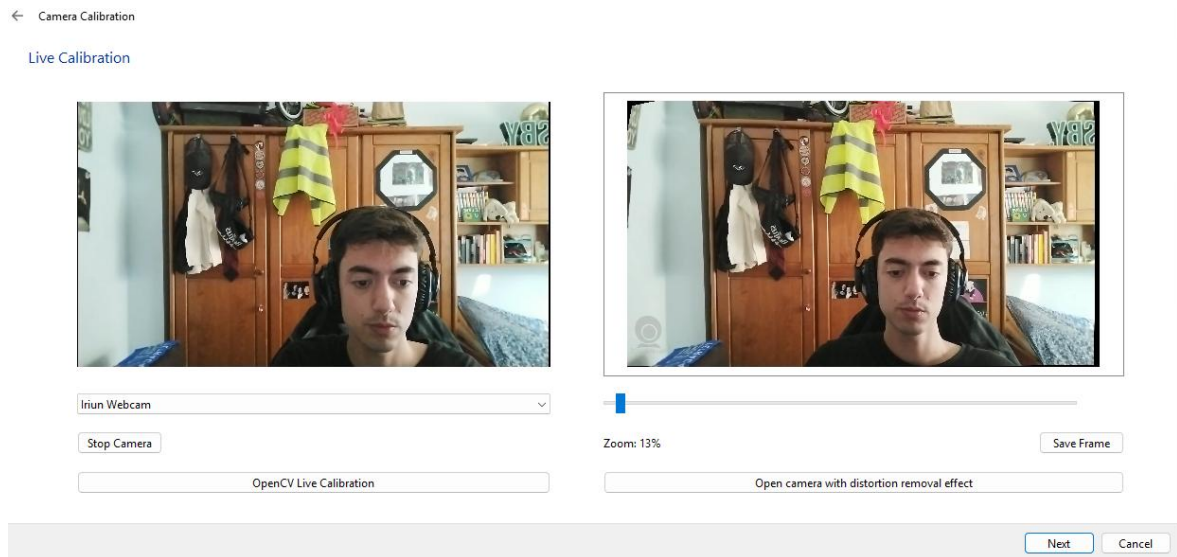


Figure 39 – Live Calibration Page Layout.

Figure 40 and Figure 41 are separate parts of the complete flowchart for the implementation of this page.

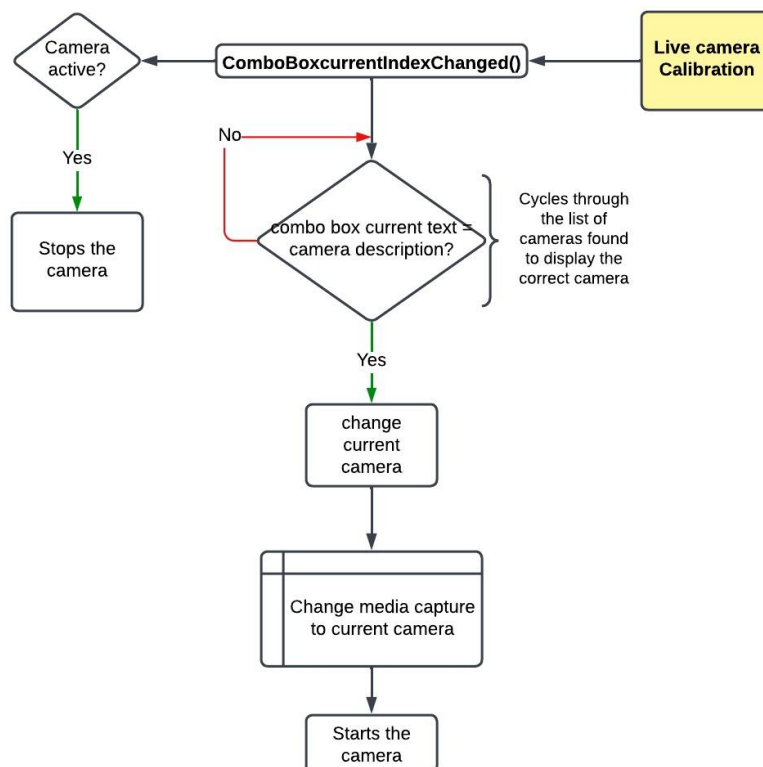


Figure 40 – Live Camera Calibration QComboBox's Functions Flowchart.

The flowchart of Figure 40 shows that when the QComboBox index changes, the program can switch the active camera through a sequence of steps. It is important to note that an index change only occurs through manual user interaction.

The first step is to check whether the current camera is active. This is done using the Qt function *QtCamera.isActive()*, which returns true or false to an IF condition in the program. Then the program cycles through the list of cameras obtained by the *getCameras()* function and attempts to find a match between the current text of the *QComboBox*'s and a camera description in the list. When a match is found, the program assumes that this is the selected camera and proceeds to display it.

A *QMediaCaptureSession* is used to display the camera. For this to work, the camera selected by the user must be assigned to the *QMediaCaptureSession*, which then displays the feed on the *QVideoWidget* present in the page layout.

In Figure 41 we can see the second part of the complete flowchart for this page. This part contemplates the *getCameras()* function, the *ParaCameraClicked()* function and the *ZoomValueChanged()* function.

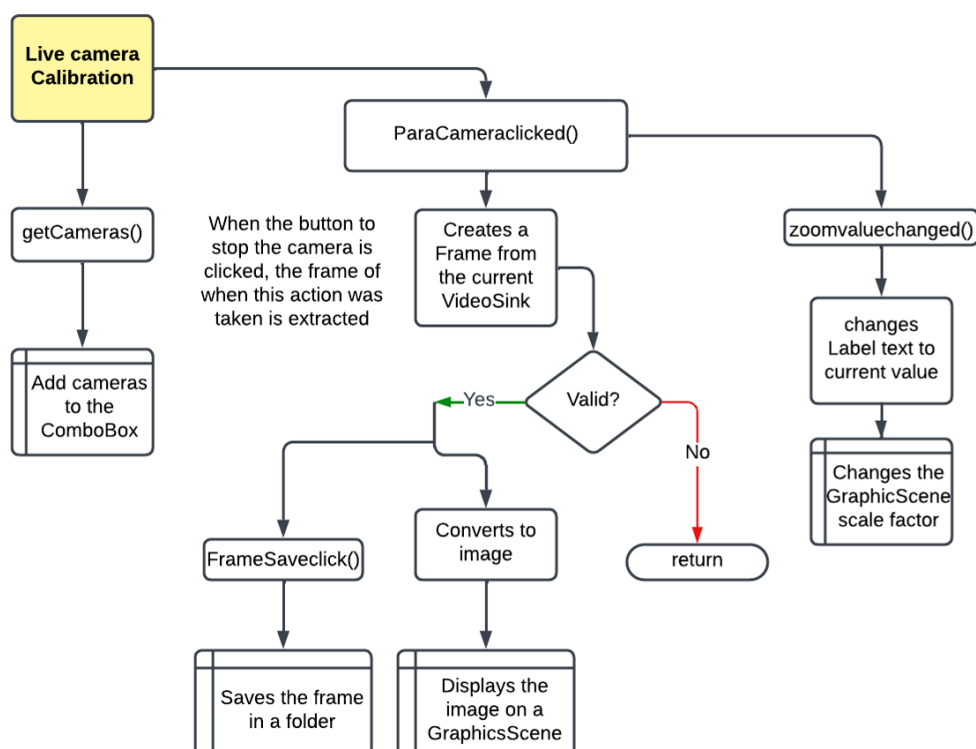


Figure 41 - Live Camera Calibration Frame Extraction Related Functions Flowchart.

The purpose of the *getCameras()* function is to search the computer where the program is running for available video devices. It then retrieves each device's description and stores them in a *QList*, which is ultimately used to populate the indexes of the *QComboBox*.

Other functions on this part of the flowchart work on the same section of this page, the frame extraction and handling. The *ParaCameraClicked()* function is activated whenever the *PushButton* to extract the frame is clicked by the user. This event is then connected to a function that extracts a frame, checks if the frame is valid, converts it to an image and displays it on the *QGraphicsScene* in the layout. The last function in analysis is the *ZoomValueChanged()*, this function is linked to the slider in the layout whose purpose is to let the user change the zoom factor in the frame display. Whenever the slider's value is changed (by altering its position) the *ZoomValueChanged()* function is triggered, then another function connected to this one takes the current zoom value, displays it on a *Label* in the program and changes the *QGraphicsScene* scale in order to correspond to the current zoom value.

Lastly the *FrameSaveclicked()* function is activated once the corresponding button is clicked and saves the Frame represented on the *QGraphicsScene*. This function saves the Frame based on the representation on the screen with the same scale factor and central point. This function works thanks to the Qt class *Painter*, and its render function.

First the scale factor of the zoom, and the portion of the screen that is being visualized is stored, Figure 42. Then a *QPainter* is associated with a *QPixmap* that corresponds to this portion of the screen and renders the final image. The results are saved to a folder in the source directory, Figure 43.

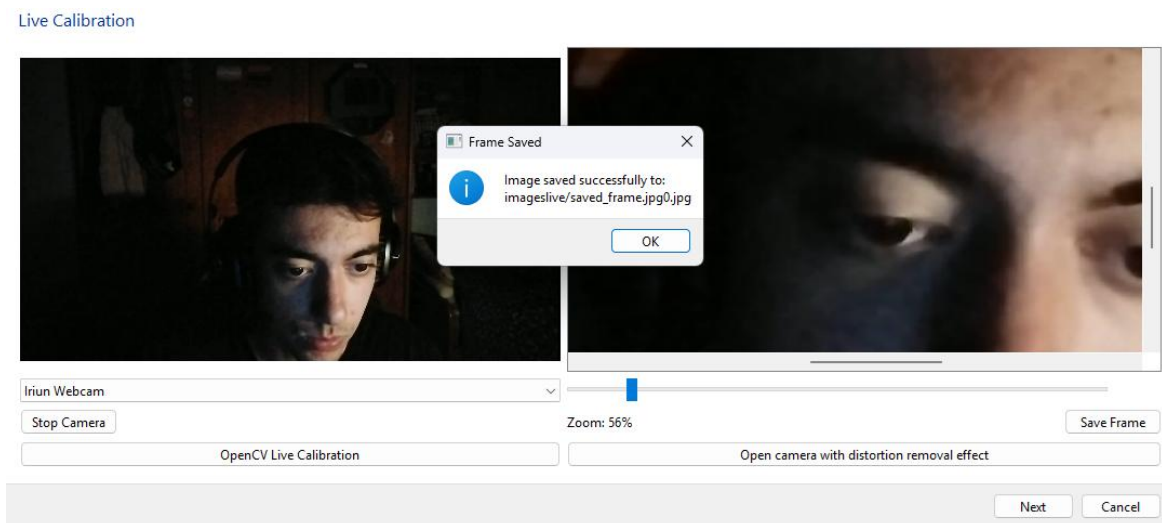


Figure 42 – Results of Clicking the Frame Save Button.

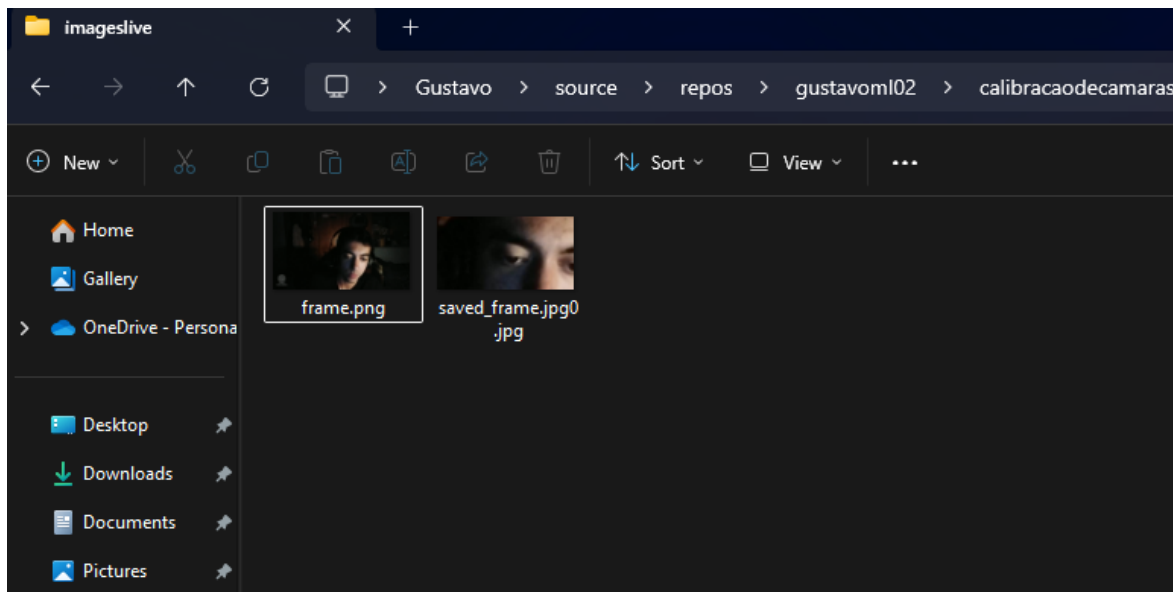


Figure 43 – Frame Saved in the Folder.

The remaining section of this page implements a calibration mode that relies exclusively on OpenCV's internal functions. This part of the interface is linked to two dedicated PushButtons: “Calibration Live” and “Undistorted Camera”. These buttons trigger distinct OpenCV routines, one performs live camera calibration by capturing multiple frames, while the other displays the camera feed with distortion correction applied in real time.

Figure 44 presents a flowchart illustrating the sequence of operations executed when either button is clicked, outlining the processes associated with each.

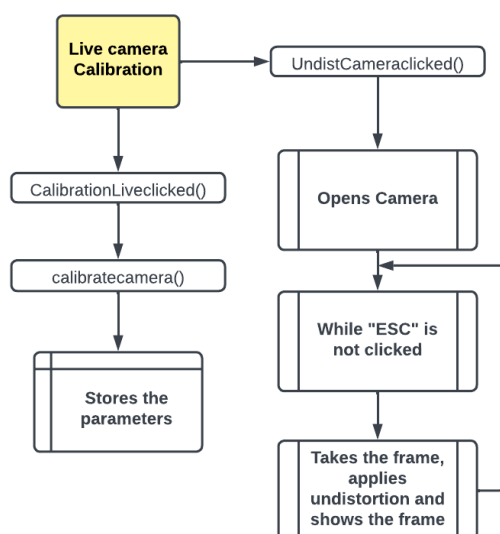


Figure 44 – Live Camera Calibration’s Calibration Related Functions.

When the Calibration Live button is clicked by the user, a function is called that tries to open the camera, and upon its success looks for a chessboard in the camera, with an associated delay of one second, so that the user can readjust the chessboard used within the latency of frames taken, Figure 45.

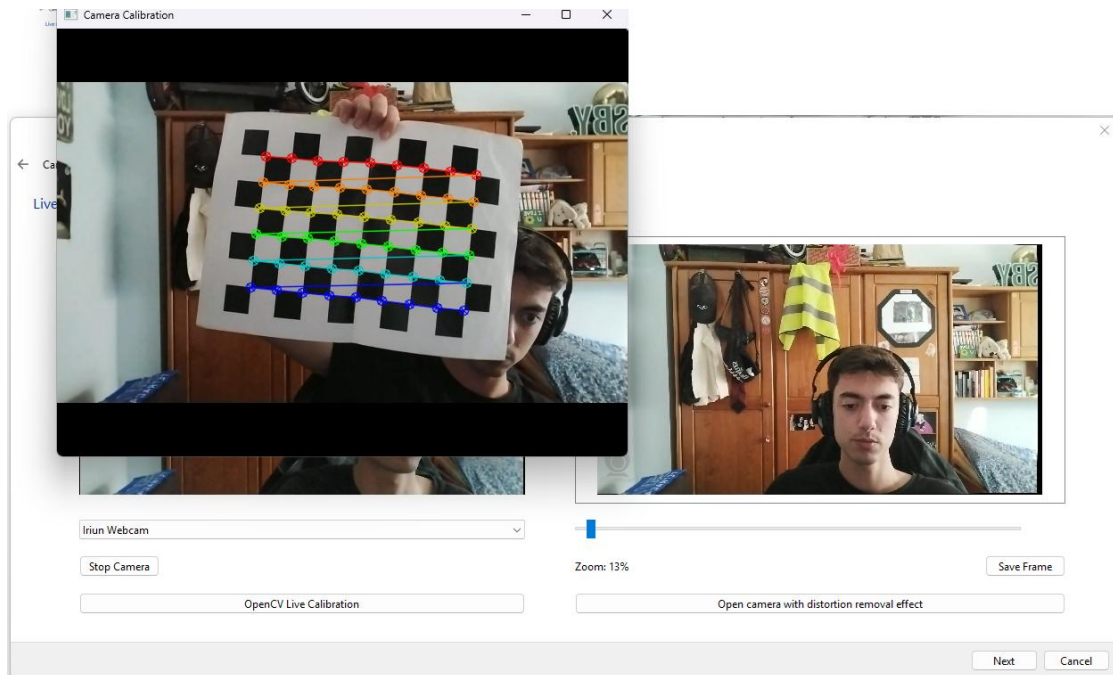


Figure 45 – Ongoing Live Calibration.

This process concludes after taking 15 frames containing a chessboard and uses the OpenCV's *calibrateCamera()* function to then extract the parameters from these frames, saving the result on a second file (different from the one featured on the Image Calibration Page).

When the user finishes this step, the Undistorted Camera button becomes available (because this button is dependent on a flag that only activates after going through the previous step), Figure 46. This button calls a function that opens the camera and at each frame captured, converts it to an undistorted frame by using the *initUndistortRectifyMap()* function as well as the *remap()* function.

This is achieved using OpenCV's *initUndistortRectifyMap()* and *remap()* functions. The *initUndistortRectifyMap()* function generates the necessary transformation maps based on the previously calculated calibration parameters. These maps are then passed to the *remap()* function, which applies the undistortion transformation to each frame as it is captured. This

pipeline allows for a live preview of the corrected camera feed, enabling users to visually validate the calibration results.

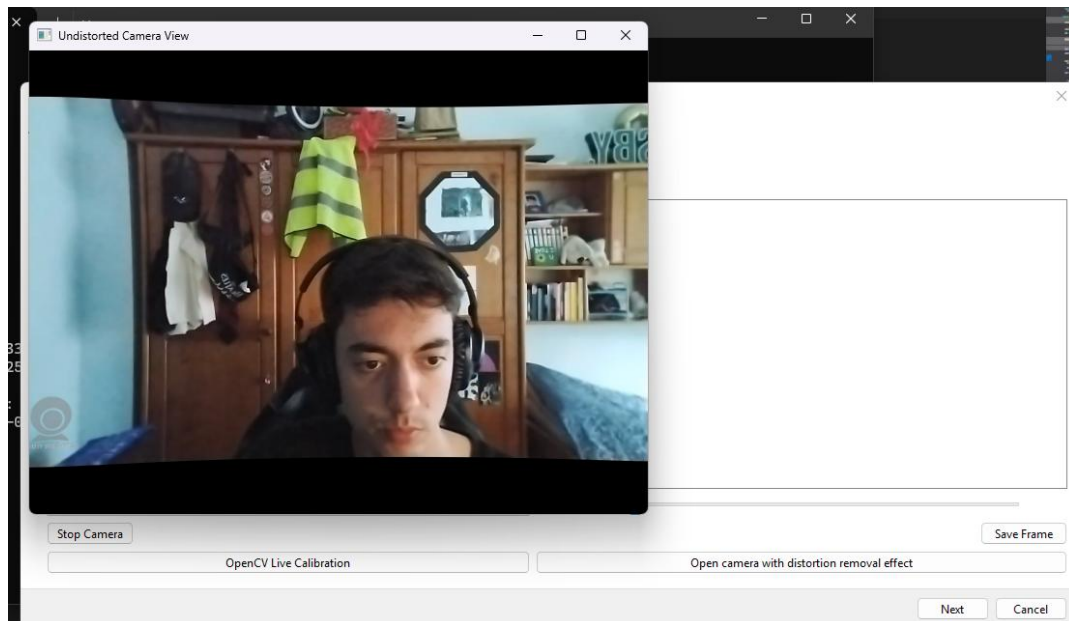


Figure 46 – Undistorted Camera Opened.

3.2.8 Conclusion Page

Completing this program is the conclusion page. This page contains only two labels that are only meant to inform the user once again of the path to which the images were saved and upon clicking the finish button, the program ends, Figure 47.

← Camera Calibration

Calibration Completed

The calibration process (images, and calibration results such as distortion coefficients and intrinsic parameters) are saved in the source folder

Click the finish button to end this wizard

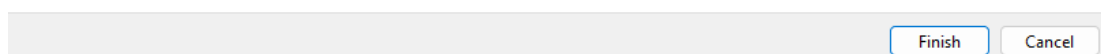


Figure 47 – Conclusion Page Layout.

4. RESULTS

4.1 Results obtained

A great variety of tests using mostly the Image Calibration Page were run using different camera angles, using more or less images and even using Instagram filters.

From these tests it was concluded that the calibration precision is highly dependent on these two main factors (considering that the number of rows, columns and square size are always valid and true):

- Number of images used
- Chessboard positions (angle, proximity and rotation)

With the increasing number of images used, the calibration becomes more and more precise, especially when it comes to the intrinsic parameters.

Varying the chessboard position is also crucial for good calibration, not only for the intrinsic parameters such as the focal point (estimated by proximity of the chessboard to the camera) as well as the extrinsic parameters (rotation and translation vectors of the camera). It's essential to cover the whole field of view of the camera with the chessboard, including the corners and edges which allows the algorithm to more effectively analyze lens distortion and generate a more reliable calibration model.

4.1.1 Normal calibration (No filters, varied images)

This section serves as an example for a normal calibration using no filters and having multiple images as references in which the chessboard has multiple positions and angles with varying distances from the camera, this is the best way to obtain a precise and complete calibration. In the Figure 48 below it's possible to see some of the images fed into the program.



Figure 48 – Examples of images uploaded for a normal calibration.

One example of the undistortion image result can be seen in Figure 49.



Figure 49 – Example of an undistorted image for a normal calibration.

We can see by the “black margins” that the program compensates for the existent distortion (mostly radial distortion) evenly meaning that the program received a good variety of samples.

The camera matrix and distortion coefficients found were as seen in Tables 2 and 3.

Table 2 – Camera Matrix obtained from a normal and successful calibration.

3217,22	0	2112,34
0	3213,88	1561,54
0	0	1

Table 3 – Distortion Coefficients obtained from a normal and successful calibration.

0,14715224	-0,57956308	-0,00109513	0,00212650	0,93670768
------------	-------------	-------------	------------	------------

4.1.2 Calibration with fewer images

When the program receives only a single image for calibration, there is often a significant deviation from the results obtained using multiple images. This occurs because the calibration algorithm cannot cross-reference and average values across several samples and must rely solely on the data provided by that one image. As a result, the estimated camera parameters are generally way less accurate and more susceptible to noise or detection errors. In Figure 50 below, we can observe the typical outcome when calibration is performed using only a single uploaded image where the board has a great inclination.

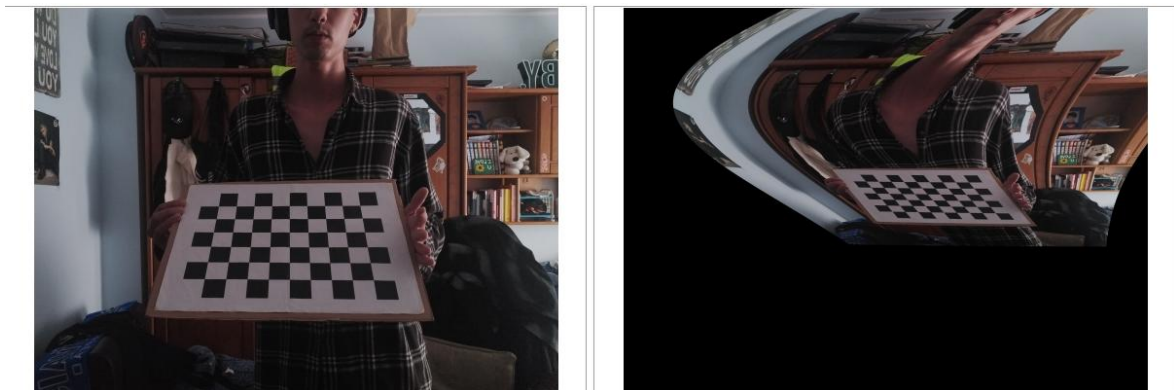


Figure 50 – Single image calibration (inclined chessboard).

Next, in Figure 51, the result for a calibration that only has an image as the reference, but the board has no inclination and is somewhat centered to the camera central point.



Figure 51 - Single image calibration (chessboard with no inclination and centered).

And finally, the results that were obtained when only uploading an image where the chessboard is rotated (seen in Figure 52).



Figure 52 - Single image calibration (chessboard with rotation, uncentered).

In some cases, calibrating with a single image can yield reasonably good results; however, introducing even just a second image significantly improves the accuracy of the calibration. This is because multiple perspectives allow the algorithm to better estimate the camera's intrinsic and extrinsic parameters, reducing sensitivity to noise or feature detection errors. Below in Figure 53 it's possible to observe the results obtained from a two-image calibration with “symmetrical” board rotation and position.

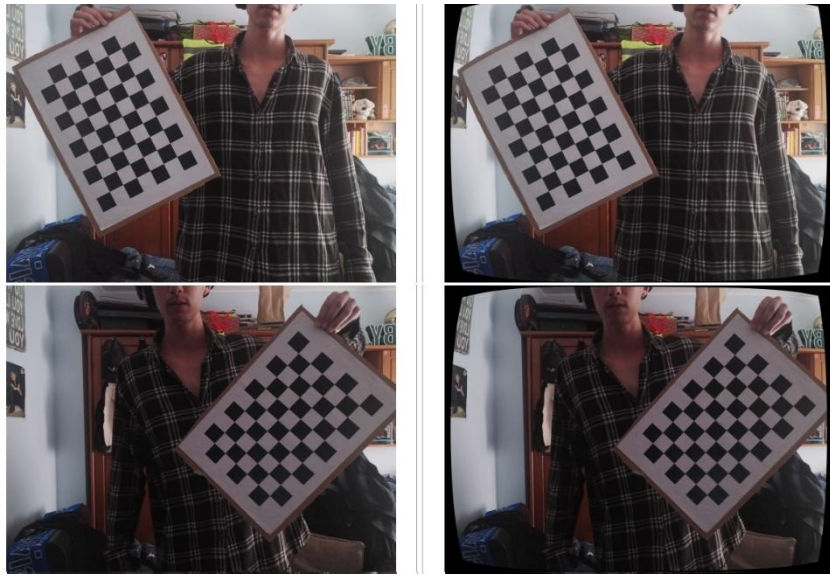


Figure 53 – Two image calibration with rotated boards.

4.1.3 Calibration with images that have reflection on the chessboard

Another test conducted on the Image Calibration Page involved the use of a printed paper chessboard. However, the surface of the ink caused light to reflect off the pattern, which significantly interfered with the calibration process. Out of the five images uploaded, only two were successfully recognized by the program, the remaining three were ignored. This issue likely occurred because the reflected light obscured the visibility of the chessboard corners, making it difficult for the algorithm to accurately detect and process the calibration pattern. The images used and the results obtained can be seen in Figure 54. Figure 55 features the console content when the *calibrateImage()* function was running.

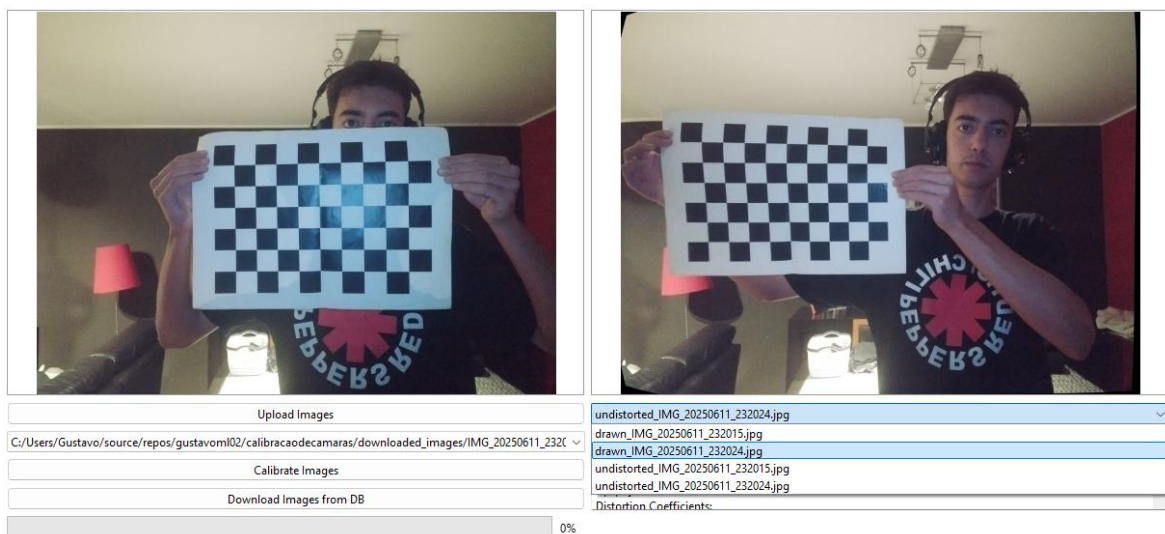


Figure 54 – Results for a calibration with light reflection.

```
images\IMG_20250611_232015.jpg
images\IMG_20250611_232024.jpg
images\IMG_20250611_232033.jpg
Warning: Chessboard not found in images\IMG_20250611_232033.jpg

images\IMG_20250611_232041.jpg
Warning: Chessboard not found in images\IMG_20250611_232041.jpg

images\IMG_20250611_232050.jpg
Warning: Chessboard not found in images\IMG_20250611_232050.jpg
Files in directory: 2

Calibration results saved to 'calibration_results.txt'.
```

Figure 55 – Chessboards not found during the light reflection calibration.

4.1.4 Calibration with images that have an *Instagram* distortion filter applied

The last test that was run was an idea that occurred while using the smartphone application Instagram. The app allows users to apply multiple filters to the smartphones camera and sometimes these filters feature some kind of distortion effect. The filter selected for this test had great distortion towards the edges, which is perfect to teste extreme cases. Some of the images uploaded into the program using this filter can be seen in Figure 56.



Figure 56 - Examples of images uploaded for a normal calibration.

One example of the undistortion image result can be seen in Figure 57.

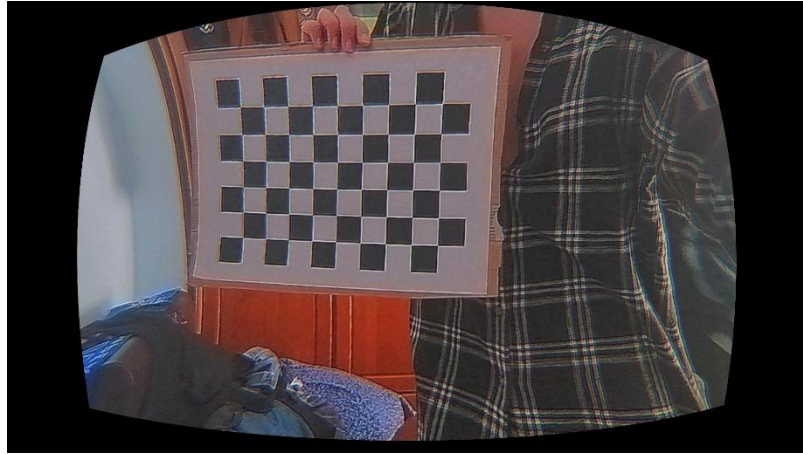


Figure 57 - Example of an undistorted image for a normal calibration.

By the “black margins” presented in the undistorted image we can see that the program compensates greatly for the existent distortion evenly meaning that the program received a good variety of samples and it’s possible to see a great correcting when comparing to the original image (first one seen in Figure 56).

4.1.4 Live Calibration

The live calibration tests were intended as a quick and convenient method for performing camera calibration. However, several challenges arose during implementation primarily due to the limitations of the camera setup used. Specifically, the camera was not directly connected to the computer running the tests, but instead streamed video wirelessly. This might have introduced significant imprecision in the calibration results.

The lack of a direct connection, combined with potential unknown image compression methods used in the wireless stream, likely degraded the image quality. As a result, the calibration process was affected by reduced frame consistency and possible artifacts, both of which can hinder accurate detection of chessboard corners and ultimately impact the precision of the computed calibration parameters. In Figure 58, we can observe the program successfully running and accurately detecting the chessboard corners, highlighting them in the process.

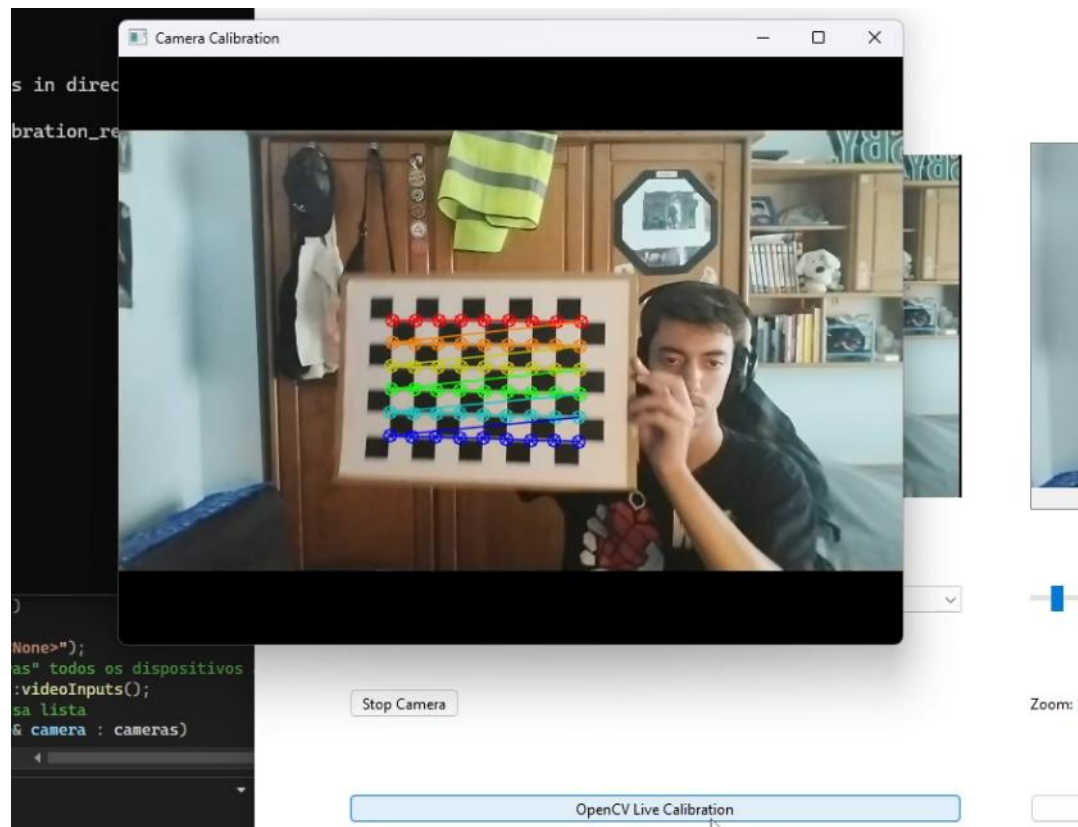


Figure 58 – Chessboard detection (live calibration results).

Figure 59 displays the results of this test. Despite having a good sample (15 frames) the results appeared to be different from the calibration using an image and were barely noticeable.

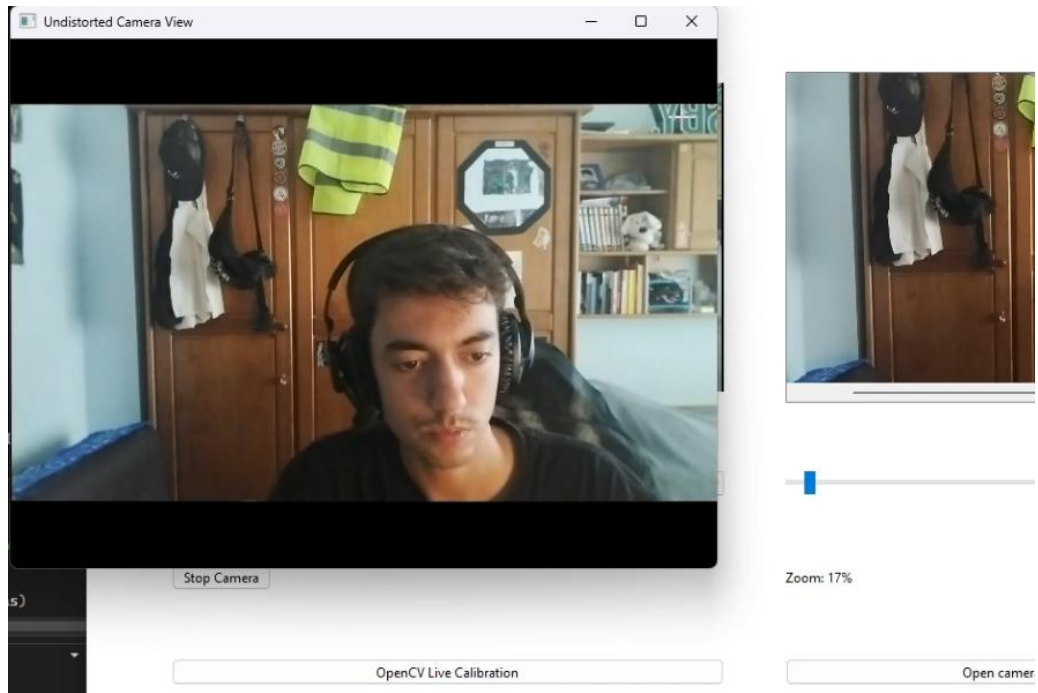


Figure 59 – Undistortion camera open.

Another method featured on the same page was to open the camera and calibrate a frame at the click of a button, using the parameters extracted from the pre-taken image calibration. These results, although more accurate than live calibration itself, still presented some imprecision as seen in Figure 60 (the parameters refer to a normal calibration using varying chessboard positions and angles featured in [Section 4.1.1](#)).

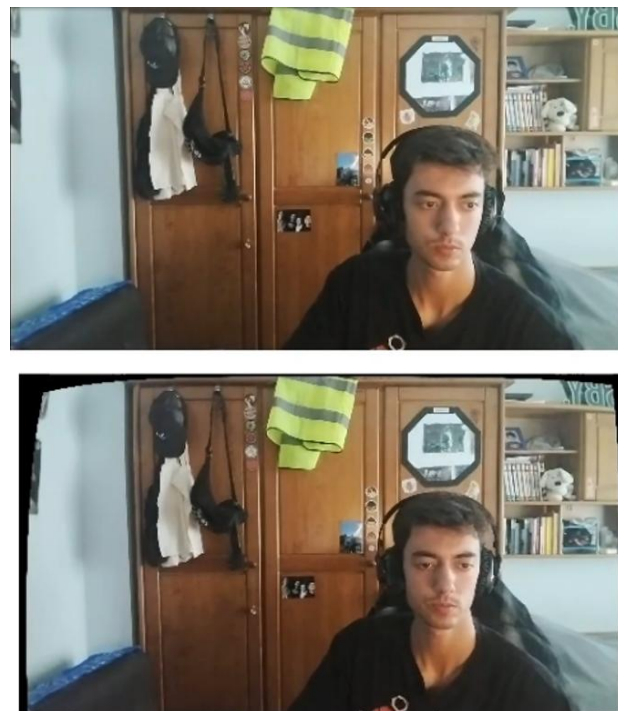


Figure 60 – Single frame calibration with live camera.

4.2 Calibration Imprecisions and Limitations

Most of the calibration imprecisions and limitations were found when using the camera matrix and distortion coefficients from one type of calibration to the other. This means that when using the parameters calculated in the Live Camera Calibration Page in the Image Calibration Page and vice versa, the results obtained were not equal. Therefore, it can be concluded that this problem probably arises from the fact that these two types of calibration are using different image quality or compression (they also use different images which could suggest a difference in results, but not as great of a deviation as the one observed).

The method of Live Camera Calibration also appeared to be greatly more imprecise than the Image Camera Calibration one.

5. CONCLUSIONS

The development of this project culminated in the successful creation of a cross-platform, user-friendly wizard application for camera calibration. The tool integrates powerful libraries, Qt for the user interface and OpenCV for image processing, to guide users through either image-based or live camera calibration in a structured and intuitive manner.

From a technical standpoint, this application introduces:

- A **modular wizard-style interface**, which helps structure the calibration workflow and simplifies the process for users, even those with little technical expertise, which was a problem Gislotica wanted to see fixed.
- A **flexible calibration backend**, capable of parsing user-specified chessboard sizes and square dimensions, storing parameters to a persistent database using SQLite3, allowing for broader user experience and improving precision.
- An **image calibration module** that automatically filters invalid images, calibrates using valid ones, and provides real-time feedback and visualization, as well as a great number of error control, allowing for a flawless flow of the program.
- A **live calibration module**, capturing real-time frames through a camera feed, with frame extraction, zooming, and OpenCV calibration support. Although, this specific part of the tool (the live calibration referring to the undistorted camera) was a bit ambitious resulting in a more imprecise calibration that varies greatly.
- A **robust and maintainable code** structure was implemented, ensuring adaptability to future enhancements or integration into Gislotica's existing systems.

Throughout development, various technical challenges such as database integration, the OpenCVs library integration as well as its functions, and calibration data parsing were successfully overcome.

5.1 Future Perspectives and Possible Enhancements to the Project

Ultimately, this application not only meets the original objective as well as most of my personal objectives of simplifying camera calibration in an industrial setting but also lays the foundation for more refined programs.

Some of the objectives that were not met were to try to use the program in an industrial setting and adjust it in a setting capable of lacking more precision. Probably due to the varying angles and camera settings, cleanliness, varying lighting, and paper chessboard that has some light reflection associated.

Some possible improvements for this project include:

- 3D visualization of the cameras position, using the extrinsic parameters as a reference. (This feature could be obtained by using, for example, MATLAB [13] and some reconstruction functions present in the OpenCV library [5]).
- Tests and possible code replacement to obtain more precision during live calibration.
- The addition of AI tools to recognize good and bad calibration patterns, allowing low experienced users to have a better understanding of the results.

Documental References

- [1] **Zhang, Z.** (2000). *Camera Calibration*. In K. Ikeuchi (Ed.), **Computer Vision** (Chapter 2). Springer. [Microsoft Research Technical Report MSR-TR-98-71].
- [2] O. Faugeras. *Three-Dimensional Computer Vision: A Geometric Viewpoint*. MIT Press, 1993.
- [3] Gruen, Armin. (2001). *Calibration and Orientation of Cameras in Computer Vision*. (Springer Series in Information Sciences, 34)
- [4] Camera calibration With OpenCV (opencv.org):
https://docs.opencv.org/4.1.0/d4/d94/tutorial_camera_calibration.html
- [5] Camera Calibration and 3D Reconstruction (opencv.org):
https://docs.opencv.org/4.x/d9/d0c/group_calib3d.html
- [6] INTRODUÇÃO AO C++ -
[HTTPS://WWW.W3SCHOOLS.COM/Cpp/Cpp_INTRO.ASP](https://www.w3schools.com/cpp/cpp_intro.asp)
- [7] QT WIKI “ABOUT QT” - [HTTPS://WIKI.QT.IO/ABOUT_QT](https://wiki.qt.io/About_QT)
- [8] SQLite “About” section: <https://sqlite.org/about.html>
- [9] OpenCV “About” section: <https://opencv.org/about/>
- [10] Kaneda, S., & Premachandra, C. (2022). *AI based object recognition performance between general camera and omnidirectional camera images*. In *Proceedings of the 2022 2nd International Conference on Image Processing and Robotics (ICIPRob)*
- [11] Kim, K., & Oh, S. (2021). *Edge computing system applying integrated object recognition based on deep learning*. In *Proceedings of the 2021 International Conference on Advanced Communications Technology (ICACT)*
- [12] Morena Alberola, Á., Molina Gallego, G., & Garay Maestre, U. (2019). *Artificial vision and language processing for robotics*. Packt Publishing.
- [13] Creating 3-D Plots in MATLAB:
<https://www.mathworks.com/help/matlab/visualize/creating-3-d-plots.html>

Anexo A. Projects Final Code (header file (.h))

```
#ifndef CALIBRACAODECAMARAS_H
#define CALIBRACAODECAMARAS_H

#include <QWizard>
#include <QCamera>
#include <QCameraDevice>
#include <QMediaDevices>
#include <QMediaCaptureSession>
#include <QVideoSink>
#include <QImage>
#include <QGraphicsScene>
#include <QPixmap>
#include <QSlider>
#include <QVideoWidget>
#include <QByteArray>
#include <QBuffer>
#include <QProgressBar>
#include "sqlite3.h"
#include <opencv2/opencv.hpp>

using namespace cv;

QT_BEGIN_NAMESPACE
class QCheckBox;
class QGroupBox;
class QComboBox;
class QLabel;
class QLineEdit;
class QRadioButton;
class QPlainTextEdit;
class QPushButton;
class QVBoxLayout;
class QFileInfo;
class QDate;
QT_END_NAMESPACE

class ClassWizard : public QWizard
{
    Q_OBJECT

public:
    ClassWizard(QWidget* parent = nullptr);

    static const int IntroPageId = 0;
    static const int CameraInfoPageId = 1;
    static const int ChoicePageId = 2;
    static const int ImagesPageId = 3;
    static const int CameraPageId = 4;
    static const int ConclusionPageId = 5;

    void accept() override;
};
```

```

class IntroPage : public QWizardPage
{
    Q_OBJECT

public:
    IntroPage(QWidget* parent = nullptr);

private:
    QLabel* label;
};

class ChoicePage : public QWizardPage
{
    Q_OBJECT

public:
    ChoicePage(QWidget* parent = nullptr);
    int nextId() const override;

private:
    QLabel* choicelabel;
    QVBoxLayout* choiceLayout;
    QRadioButton* imagesPageRadioButton;
    QRadioButton* cameraPageRadioButton;
};

class CameraInfoPage : public QWizardPage
{
    Q_OBJECT

public:
    CameraInfoPage(QWidget* parent = nullptr);

    void uploadfile();
    int initializedb();
    void closedb();
    void uploadtodb(QString contents, QString filename, QString date);

private:
    QLabel* introtofile;
    QLabel* boardwidthlbl;
    QLabel* boardheightlbl;
    QLabel* squaresizelbl;
    QLineEdit* boardwidthle;
    QLineEdit* boardheightle;
    QLineEdit* squaresizele;
    QPlainTextEdit* filecont;
    QPushButton* uploadButton;
    sqlite3* db;
};

class ImagesPage : public QWizardPage
{
    Q_OBJECT

public:
    ImagesPage(QWidget* parent = nullptr);
    int nextId() const override;

private slots:

```

```

    void uploadImage(QLabel* image, QComboBox* selector, QList<QPixmap>&
list);
    void displaySelectedImage(int index, QLabel* image, QList<QPixmap>&
list);
    void calibrateImage();
    int initializedb();
    QByteArray readImageFile(const QString& filePath);
    void closedb();
    void downloadImagesToFolder(const QString& dir);
    void uploadtodb(QByteArray imageData, QString imageName, QString
date);
    void uploadUndist(QLabel* image, QComboBox* selector, QList<QPixmap>&
list, QString dir);
    //void calibratedclicked()

private:
    QLabel* imageLabel;
    QLabel* referenceLabel;
    QPushButton* uploadButton;
    QPushButton* calibrationButton;
    QPushButton* downloadButton;
    QComboBox* imageSelector;
    QComboBox* refimageSelector;
    QList<QPixmap> images;
    QList<QPixmap> refs;
    QPlainTextEdit* info;
    QProgressBar* progressBar;
    sqlite3* db;
};

class CameraPage : public QWizardPage
{
    Q_OBJECT

public:
    CameraPage(QWidget* parent = nullptr);
    QList<QCameraDevice> cameras;
    QCamera* currentcam;
    QMediaCaptureSession* mediacapture;
    QVideoSink* qvs;
    QGraphicsScene* scene;
    QSlider* zoom;
    QPushButton* paracamara;
    QPushButton* gridbutton;
    QPushButton* saveframe;
    QComboBox* combocamaras;

    QVideoWidget* videoWidget;
    QLabel* zoompx;
    QLabel* zoomlab;
    QGraphicsView* framedisplay;
    QPushButton* livecalbutton;
    QPushButton* undistbutton;

private slots:
    void getCameras();
    void selectcamera(int index);
    void on_zoom_valueChanged();
    void on_paracamara_clicked();
    void on_saveframe_clicked();
    float livecalibration();

```

```

        void undistcamera();

private:
    QLabel* label;
};

class ConclusionPage : public QWizardPage
{
    Q_OBJECT

public:
    ConclusionPage(QWidget* parent = nullptr);

protected:
    void initializePage() override;

private:
    QLabel* label;
};

#endif

```

Anexo B. Projects Final Code (.cpp file)

```
#include <QtWidgets>
#include "calibracaodecamaras.h"
#include <iostream>
#include <locale>
#include <opencv2/opencv.hpp>
#include <opencv2/calib3d.hpp>
#include <opencv2/imgproc.hpp>
#include <opencv2/highgui.hpp>
#include <vector>
#include <filesystem>
#include <fstream>
#include <locale.h>
#include <regex>
#include <sstream>

using namespace cv;
using namespace std;
namespace fs = std::filesystem;

int boardWidth = 9;
int boardHeight = 6;
Size boardSize(boardWidth, boardHeight);
float squareSize = 3.5; // Chessboard square size
int flaglivecam = 0;
int flagpar = 0;
int flagupload = 0;

vector<vector<Point3f>> objectPoints;
vector<vector<Point2f>> imagePoints;
vector<Point3f> objp;
Mat cameraMatrix, distCoeffs;
Mat cameraMatrix2, distCoeffs2;
Mat cameraMatrix3, distCoeffs3;

QImage MatToQImage(const cv::Mat& mat) {
    switch (mat.type()) {
        // 8-bit, 3-channel (color image)
        case CV_8UC3: {
            QImage image(mat.data, mat.cols, mat.rows, mat.step,
                QImage::Format_RGB888);
            return image.rgbSwapped(); // Convert BGR (OpenCV) to RGB (Qt)
        }

        // 8-bit, 1-channel (grayscale image)
        case CV_8UC1: {
            QVector<QRgb> colorTable;
            for (int i = 0; i < 256; i++)
                colorTable.push_back(qRgb(i, i, i));

            QImage image(mat.data, mat.cols, mat.rows, mat.step,
                QImage::Format_Indexed8);
            image.setColorTable(colorTable);
            return image;
        }

        // 8-bit, 4-channel (e.g., BGRA)
    }
```



```

        case CV_8UC4: {
            QImage image(mat.data, mat.cols, mat.rows, mat.step,
QImage::Format_ARGB32);
            return image;
        }

        default:
            qWarning("Unsupported cv::Mat format for QImage conversion");
            return QImage();
    }
}

bool readCalibrationFile(const std::string& filename, Mat& cameraMatrix2,
Mat& distCoeffs2) {
    ifstream file(filename);
    if (!file.is_open()) {
        cerr << "Failed to open " << filename << endl;
        return false;
    }

    string line;
    string cameraBlock, distBlock;
    bool readingCamera = false, readingDist = false;

    while (getline(file, line)) {
        // Replace commas with dots (decimal formatting)
        replace(line.begin(), line.end(), ',', '.');

        if (line.find("Camera Matrix:") != string::npos) {
            readingCamera = true;
            readingDist = false;
            continue;
        }
        else if (line.find("Distortion Coefficients:") != string::npos) {
            readingCamera = false;
            readingDist = true;
            continue;
        }

        if (readingCamera) {
            cameraBlock += line;
        }
        else if (readingDist) {
            distBlock += line;
        }
    }

    // Helper lambda to extract numbers from string
    auto extractNumbers = [](const string& str) {
        vector<double> numbers;
        regex numRegex(R"([+-]?\d*\.\d+(?:[eE][+-]?\d+)?)");
        sregex_iterator it(str.begin(), str.end(), numRegex);
        sregex_iterator end;
        while (it != end) {
            numbers.push_back(stod(it->str()));
            ++it;
        }
        return numbers;
    };

    vector<double> camVals = extractNumbers(cameraBlock);

```

```

vector<double> distVals = extractNumbers(distBlock);

if (camVals.size() != 9) {
    cerr << "Error: Invalid camera matrix size (" << camVals.size()
<< " values found)." << endl;
    return false;
}

cameraMatrix2 = Mat(3, 3, CV_64F, camVals.data()).clone();
distCoeffs2 = Mat(distVals, true).reshape(1, 1); // 1 row
distCoeffs2.convertTo(distCoeffs2, CV_64F); // Reshape to a single
row

return true;
}

ClassWizard::ClassWizard(QWidget* parent)
: QWizard(parent)
{
    addPage(new IntroPage);
    addPage(new CameraInfoPage);
    addPage(new ChoicePage);
    addPage(new ImagesPage);
    addPage(new CameraPage);
    addPage(new ConclusionPage);

    setWindowTitle("Camera Calibration");
}

void ClassWizard::accept()
{
    QDialog::accept();
}

IntroPage::IntroPage(QWidget* parent)
: QWizardPage(parent)
{
    readCalibrationFile("calibration_results.txt", cameraMatrix,
distCoeffs);
    readCalibrationFile("calibration_resultslive.txt", cameraMatrix2,
distCoeffs2);

    std::locale::global(std::locale("pt_PT.UTF-8"));

    setTitle(u8"Wizard for camera calibration");
    setSubTitle(u8"\nWelcome to the camera calibration wizard. This
wizard will guide you through the process of calibrating a camera using
either images or a live camera feed.");

    label = new QLabel("\n All data is saved in a database in the
program's source folder.\n By default the program uses the last
parameters saved on the file calibration_results.txt.");
    label->setWordWrap(true);

    QVBoxLayout* layout = new QVBoxLayout;
    layout->addWidget(label);
    setLayout(layout);
}

ChoicePage::ChoicePage(QWidget* parent)

```

```

        : QWizardPage(parent)
    {
        setTitle("Wizard for Camera Calibration");

        choicelabel = new QLabel("Choose between live calibration and image
calibration:");
        imagesPageRadioButton = new QRadioButton(tr("Image Calibration"));
        cameraPageRadioButton = new QRadioButton(tr("Live Calibration"));

        QVBoxLayout* choiceLayout = new QVBoxLayout;
        choiceLayout->addWidget(choicelabel);
        choiceLayout->addWidget(imagesPageRadioButton);
        choiceLayout->addWidget(cameraPageRadioButton);

        setLayout(choiceLayout);
    }
    int ChoicePage::nextId() const
    {
        if (imagesPageRadioButton->isChecked()) {
            return ClassWizard::ImagesPageId;
        }
        else if (cameraPageRadioButton->isChecked()) {
            return ClassWizard::CameraPageId;
        }
        return ClassWizard::CameraInfoPageId; // Default to CameraInfoPageId
    }
    if none selected
}

CameraInfoPage::CameraInfoPage(QWidget* parent)
    : QWizardPage(parent), db(nullptr)
{
    setTitle("Camera Information");
    introtofile = new QLabel("To begin, upload the file with the camera
information that will be relevant for the calibration and select the
number of rows, columns and square size.\n This program will only
consider these parameters valid for the Image Calibration Page, and will
use the inputed parameters only once.\n If no information is inputed, the
program will continue with default values and create new parameters for
calibration:");
    boardheightlbl = new QLabel("Board Height (rows):");
    boardwidthlbl = new QLabel("Board Width (columns):");
    squaresizelbl = new QLabel("Square Size (in cm):");
    boardheightle = new QLineEdit("6");
    boardheightle->setValidator(new QIntValidator(0, 100, this));
    boardwidthle = new QLineEdit("9");
    boardwidthle->setValidator(new QIntValidator(0, 100, this));
    squaresizele = new QLineEdit("3.5");
    squaresizele->setValidator(new QDoubleValidator(0.0, 100.0, 2,
this));
    filecont = new QPlainTextEdit();
    uploadButton = new QPushButton("Upload file");

    registerField("filecontent", filecont);

    connect(uploadButton, &QPushButton::clicked, this,
&CameraInfoPage::uploadfile);
    connect(boardheightle, &QLineEdit::textChanged, [](const QString&
text) {
        boardHeight = text.toInt();
        boardSize = Size(boardWidth, boardHeight);
    });

```

```

        cout << "Board Size: " << boardSize.width << "x" <<
boardSize.height << endl;
    });
    connect(boardwidthhle, &QLineEdit::textChanged, [](const QString&
text) {
        boardWidth = text.toInt();
        boardSize = Size(boardWidth, boardHeight);
        cout << "Board Size: " << boardSize.width << "x" <<
boardSize.height << endl;
    });
    connect(squaresizele, &QLineEdit::textChanged, [](const QString&
text) {
        squareSize = text.toFloat();
        cout << "Square Size: " << squareSize << " cm" << endl;
    });

    QGridLayout* layout = new QGridLayout;
    layout->addWidget(introtofile, 0, 0, 1, 2);
    layout->addWidget(filecont, 1, 0, 1, 2);
    layout->addWidget(uploadButton, 2, 0, 1, 2);
    layout->addWidget(boardheightlbl, 3, 0);
    layout->addWidget(boardheightle, 3, 1, Qt::AlignRight);
    layout->addWidget(boardwidthlbl, 4, 0);
    layout->addWidget(boardwidthhle, 4, 1, Qt::AlignRight);
    layout->addWidget(squaresizelbl, 5, 0);
    layout->addWidget(squaresizele, 5, 1, Qt::AlignRight);

    setLayout(layout);
}

void CameraInfoPage::uploadfile()
{
    QString filePath = QFileDialog::getOpenFileName(nullptr, "Open Text
File", QString(), "Text Files (*.txt)");
    QString contents;
    string filePathStr = filePath.toStdString();

    /*if (readCalibrationFile(filePathStr, cameraMatrix, distCoeffs) !=
true) {
        filecont->setPlainText("Error: Calibration File not in the
current format ensure that it reads like in this example:\nCamera
Matrix:\n[3317, 651144705852, 0, 2014, 318741125316\n0, 3316,
035857955561, 1674, 495091022121\n0, 0, 1]\nDistortion Coefficients
:\n[0, 2132400195360207, -1, 203487041767592, 0, 002274263018278898, -0,
008488002293880892, 2, 165148949643965]\n");
        QMessageBox::warning(this, "Camera Info upload failed", "Please
ensure that the file is in the correct format as seen in the textbox");
        return;
    }

    readCalibrationFile(filePathStr, cameraMatrix, distCoeffs);*/

    QMessageBox::information(this, "Camera Info uploaded sucessfully!",
"File successfully uploaded and coeffs saved:\n" + filePath);

    if (!filePath.isEmpty()) {
        QFile file(filePath);
        if (file.open(QIODevice::ReadOnly | QIODevice::Text)) {
            contents = file.readAll();
            file.close();
            QFileInfo filename(filePath);

```

```

        filePath = filename.fileName();
        QDate date(QDate::currentDate());
        initializeddb();
        uploadtodb(contents, filePath, date.toString("yyyy-MM-dd"));
        closedb();
        filecont->setPlainText(contents);
    }
    else {
        filecont->setPlainText("Error: Unable to open the file.");
    }
}
else {
    filecont->setPlainText("File selection canceled.");
}
filecont->show();
flagpar = 1;
}

int CameraInfoPage::initializeddb()
{
    int result = sqlite3_open("uploadsdatabase.db", &db);
    if (result) {
        std::cerr << "Can't open database: " << sqlite3_errmsg(db) <<
std::endl;
        return 1;
    }
    else {
        const char* sqlCreateTable = "CREATE TABLE IF NOT EXISTS
camerainfo ("
            "id INTEGER PRIMARY KEY AUTOINCREMENT,"
            "date_of_upload TEXT NOT NULL,"
            "file_name TEXT NOT NULL,"
            "contents BLOB NOT NULL);";
        char* errorMessage = 0;
        result = sqlite3_exec(db, sqlCreateTable, 0, 0, &errorMessage);
        if (result != SQLITE_OK) {
            std::cerr << "SQL error: " << errorMessage << std::endl;
            sqlite3_free(errorMessage);
        }
    }
    return result;
}

void CameraInfoPage::uploadtodb(QString contents, QString filename,
QString date)
{
    QString sqlInsert = "INSERT INTO camerainfo (date_of_upload,
file_name, contents) VALUES ('" +
        date + "', '" + filename + "', '" + contents + "');";

    // Convert the QString to a UTF-8 encoded QByteArray
    QByteArray byteArray = sqlInsert.toUtf8();

    // Execute the SQL query using sqlite3_exec
    int rc = sqlite3_exec(db, byteArray.constData(), nullptr, nullptr,
nullptr);

    if (rc != SQLITE_OK) {
        std::cerr << "Failed to execute statement: " <<
sqlite3_errmsg(db) << std::endl;
    }
}

```

```

        qDebug() << "Failed to execute statement: " <<
sqlite3_errmsg(db);
    }
}

void CameraInfoPage::closedb()
{
    if (db) {
        sqlite3_close(db);
        db = nullptr;
    }
}

ImagesPage::ImagesPage(QWidget* parent)
: QWizardPage(parent), db(nullptr)
{
    setTitle("Upload imagens for calibration");

    setSubTitle("Upload images for calibration. When using this method of
calibration, upload images or download previous uploads from the database
to proceed with the calibration\n\n"
        "After uploading images, simply click the 'Calibrate Images'
button to start the calibration process. The results will appear to the
right of the uploaded images\n\n"
        "The images will be saved in the 'images' folder and uploaded to
the database.\n\n"
        "You can also download the images from the database to a folder
of your choice.");

    imageLabel = new QLabel;
    imageLabel->setAlignment(Qt::AlignCenter);
    imageLabel->setFrameStyle(QFrame::Box | QFrame::Sunken);
    imageLabel->setFixedSize(600, 400);

    referenceLabel = new QLabel;
    referenceLabel->setAlignment(Qt::AlignCenter);
    referenceLabel->setFrameStyle(QFrame::Box | QFrame::Sunken);
    referenceLabel->setFixedSize(600, 400);

    progressBar = new QProgressBar;
    int pb;
    progressBar->setRange(0, 100);    // Set min/max
    progressBar->setValue(0);

    uploadButton = new QPushButton("Upload Images");
    downloadButton = new QPushButton("Download Images from DB");
    imageSelector = new QComboBox;

    refimageSelector = new QComboBox;

    calibrationButton = new QPushButton("Calibrate Images");

    info = new QPlainTextEdit;
    info->setReadOnly(1);

    connect(uploadButton, &QPushButton::clicked,
std::bind(&ImagesPage::uploadImage, this, imageLabel, imageSelector,
std::ref(images)));

```

```

        connect(downloadButton, &QPushButton::clicked,
std::bind(&ImagesPage::downloadImagesToFolder, this,
"downloaded_images/"));
        connect(imageSelector,
QOverload<int>::of(&QComboBox::currentIndexChanged),
std::bind(&ImagesPage::displaySelectedImage, this, std::placeholders::_1,
imageLabel, std::ref(images)));

        connect(refimageSelector,
QOverload<int>::of(&QComboBox::currentIndexChanged),
std::bind(&ImagesPage::displaySelectedImage, this, std::placeholders::_1,
referenceLabel, std::ref(refs)));

        connect(calibrationButton, &QPushButton::clicked,
std::bind(&ImagesPage::calibrateImage, this));

        QGridLayout* layout = new QGridLayout;
        layout->addWidget(imageLabel, 0, 0);
        layout->addWidget(uploadButton, 1, 0);
        layout->addWidget(imageSelector, 2, 0);

        layout->addWidget(referenceLabel, 0, 1);
        layout->addWidget(refimageSelector, 1, 1);

        layout->addWidget(calibrationButton, 3, 0);
        layout->addWidget(progressBar, 5, 0);
        layout->addWidget(downloadButton, 4, 0);

        layout->addWidget(info, 2, 1, 3, 1);
        setLayout(layout);
    }

    int ImagesPage::nextId() const
    {
        return ClassWizard::ConclusionPageId;
    }

    void ImagesPage::uploadImage(QLabel* image, QComboBox* selector,
QList<QPixmap>& list)
    {
        selector->clear();
        list.clear();
        image->clear();

        QStringList filePaths = QFileDialog::getOpenFileNames(nullptr, "Open
Image Files", QString(), "Image Files (*.png *.jpg *.bmp)");

        fs::remove_all("images/");
        if (!fs::exists("images/")) {
            fs::create_directory("images/");
        }

        if (!filePaths.isEmpty()) {
            flagupload = 1;
            foreach(const QString & filePat, filePaths) {

                QPixmap imaget(filePat);
                if (!imaget.isNull()) {
                    list.append(imaget);
                    selector->addItem(filePat);
                }
            }
        }
    }

```

```

        QByteArray imageData = readImageFile(filePat);
        QFileInfo fileInfo(filePat);
        QString imageName = fileInfo.fileName();
        QDate date(QDate::currentDate());

        fs::copy(filePat.toStdString(), "images/" +
fileInfo.fileName().toStdString(), fs::copy_options::overwrite_existing);

        initializedb();
        uploadtodb(imageData, imageName, date.toString("yyyy-MM-
dd"));
        closedb();
    }
    else {
        image->setText("Error: Unable to load one or more
images.");
    }
}
if (!list.isEmpty()) {
    displaySelectedImage(0, image, list);
}
}
else {
    image->setText("File selection canceled.");
}
}

void ImagesPage::uploadUndist(QLabel* image, QComboBox* selector,
QList<QPixmap>& list, QString dir)
{
    QDir directory(dir);
    QStringList filePaths = directory.entryList(QDir::Files);

    cout << "Files in directory: " << filePaths.size() << endl;

    if (!filePaths.isEmpty()) {
        foreach(const QString & filePat, filePaths) {

            QString fullPath = directory.absoluteFilePath(filePat);
            QPixmap imaget(fullPath);
            if (!imaget.isNull()) {
                list.append(imaget);
                selector->addItem(filePat);

                QByteArray imageData = readImageFile(fullPath);
                QFileInfo fileInfo(fullPath);
                QString imageName = fileInfo.fileName();
                QDate date(QDate::currentDate());

                //fs::copy(filePat.toStdString(), "images/" +
fileInfo.fileName().toStdString(), fs::copy_options::overwrite_existing);

                //initializedb();
                //uploadtodb(imageData, imageName/*, date.toString("yyyy-
MM-dd")*/);
                //closedb();
            }
            else {

```



```

        image->setText("Error: Unable to load one or more
images.");
    }
    }
    if (!list.isEmpty()) {
        displaySelectedImage(0, image, list);
    }
}
else {
    image->setText("File selection canceled.");
}
}

QByteArray ImagesPage::readImageFile(const QString& filePath) {
    QFile file(filePath);
    if (!file.open(QIODevice::ReadOnly)) {
        qDebug() << "Failed to open file:" << filePath;
        return QByteArray();
    }
    return file.readAll();
}

void ImagesPage::displaySelectedImage(int index, QLabel* image,
QList<QPixmap>& list)
{
    if (index >= 0 && index < list.size()) {
        image->setPixmap(list.at(index).scaled(imageLabel->size(),
Qt::KeepAspectRatio, Qt::SmoothTransformation));
    }
}

int ImagesPage::initializedb(){
    int result = sqlite3_open("uploadsdatabase.db", &db);
    if (result) {
        std::cerr << "Can't open database: " << sqlite3_errmsg(db) <<
std::endl;
        return 1;
    }
    else {
        const char* sqlCreateTable = "CREATE TABLE IF NOT EXISTS
imagesforcal ("
        "id INTEGER PRIMARY KEY AUTOINCREMENT,"
        "date_of_upload TEXT NOT NULL,"
        "file_name TEXT NOT NULL,"
        "contents BLOB NOT NULL);";
        char* errorMessage = 0;
        result = sqlite3_exec(db, sqlCreateTable, 0, 0, &errorMessage);
        if (result != SQLITE_OK) {
            std::cerr << "SQL error: " << errorMessage << std::endl;
            sqlite3_free(errorMessage);
        }
    }
    return result;
}

void ImagesPage::downloadImagesToFolder(const QString& targetDir)
{
    initializedb();
    // Create target directory if it doesn't exist
    QDir dir(targetDir);
    if (!dir.exists()) {

```

```

        dir.mkpath(".");
    }

    const char* sqlSelect = "SELECT file_name, contents FROM
imagesforcal;";
    sqlite3_stmt* stmt;

    int rc = sqlite3_prepare_v2(db, sqlSelect, -1, &stmt, nullptr);
    if (rc != SQLITE_OK) {
        std::cerr << "Failed to prepare SELECT statement: " <<
sqlite3_errmsg(db) << std::endl;
        return;
    }

    while (sqlite3_step(stmt) == SQLITE_ROW) {
        QString fileName = reinterpret_cast<const
char*>(sqlite3_column_text(stmt, 0));
        const void* blobData = sqlite3_column_blob(stmt, 1);
        int blobSize = sqlite3_column_bytes(stmt, 1);

        QByteArray imageData(static_cast<const char*>(blobData),
blobSize);

        QFile file(dir.filePath(fileName));
        if (file.open(QIODevice::WriteOnly)) {
            file.write(imageData);
            file.close();
        }
        else {
            qDebug() << "Failed to write image:" << fileName;
        }
    }

    sqlite3_finalize(stmt);

    qDebug() << "Images downloaded to:" << dir.absolutePath();
    QMessageBox::information(this, "Download Images", "Images
downloaded successfully to " + dir.absolutePath());
    closedb();
}

void ImagesPage::uploadtodb(QByteArray imageData, QString imageName,
QString date)
{
    const char* sqlInsert = "INSERT INTO imagesforcal (date_of_upload,
file_name, contents) VALUES (?, ?, ?);";
    sqlite3_stmt* stmt;

    int rc = sqlite3_prepare_v2(db, sqlInsert, -1, &stmt, nullptr);
    if (rc != SQLITE_OK) {
        std::cerr << "Failed to prepare statement: " <<
sqlite3_errmsg(db) << std::endl;
        return;
    }

    sqlite3_bind_text(stmt, 1, date.toUtf8().constData(), -1,
SQLITE_STATIC);
    sqlite3_bind_text(stmt, 2, imageName.toUtf8().constData(), -1,
SQLITE_STATIC);

```

```

        sqlite3_bind_blob(stmt, 3, imageData.constData(), imageData.size(),
        SQLITE_STATIC);

        rc = sqlite3_step(stmt);
        if (rc != SQLITE_DONE) {
            std::cerr << "Failed to execute statement: " <<
        sqlite3_errmsg(db) << std::endl;
        }

        sqlite3_finalize(stmt);
    }

void ImagesPage::calibrateImage()
{
    if (flagupload == 0)
    {
        QMessageBox::warning(this, "Camera Calibration Missing Images",
        "Please upload images before calibration");
        return;
    }

    objp.clear();
    objectPoints.clear();
    imagePoints.clear();
    cv::setBreakOnError(true);
    int pb;

    for (int i = 0; i < boardHeight; i++) {
        for (int j = 0; j < boardWidth; j++) {
            objp.push_back(Point3f(j * squareSize, i * squareSize, 0));
        }
    }
    progressBar->setValue(20);

    vector<string> imageFiles;

    for (const auto& entry : fs::directory_iterator("images")) {
        if (entry.path().extension() == ".jpg" ||
        entry.path().extension() == ".png") {
            imageFiles.push_back(entry.path().string());
            cout << "\n";
            cout << entry.path().string() << endl;
        }
    }
    cout << "\n";
    cout << imageFiles.size() << endl;

    if (imageFiles.empty()) {
        cerr << "\nError: No images found in 'images' directory." <<
    endl;
        QMessageBox::warning(this, "Camera Calibration Missing Images",
        "Please upload images before calibration");
    }

    fs::remove_all("chessboard_images/");
    if (!fs::exists("chessboard_images/")) {
        fs::create_directory("chessboard_images/");
    }
    progressBar->setValue(30);

    Mat frame, gray;

```

```

progressBar->setValue(70);
pb = 70;

for (const auto& filename : imageFiles) {
    progressBar->setValue(pb++);
    cout << "\n";
    cout << filename;
    frame = cv::imread(filename);
    if (frame.empty()) {
        cerr << "\nWarning: Could not read " << filename << endl;
        continue;
    }

    cvtColor(frame, gray, COLOR_BGR2GRAY);
    vector<Point2f> corners;
    bool found = findChessboardCorners(gray, boardSize, corners,
        CALIB_CB_ADAPTIVE_THRESH | CALIB_CB_FAST_CHECK |
        CALIB_CB_NORMALIZE_IMAGE);

    if (!found) {
        cerr << "\nWarning: Chessboard not found in " << filename <<
endl;
        fs::remove(filename); // Remove the image if chessboard is
not found
        continue; // Ignora a imagem sem chessboard
    }

    if (found) {
        cornerSubPix(gray, corners, Size(11, 11), Size(-1, -1),
            TermCriteria(TermCriteria::EPS + TermCriteria::COUNT, 30,
0.001));

        imagePoints.push_back(corners);
        objectPoints.push_back(objp);

        drawChessboardCorners(frame, boardSize, corners, found);

        string outputFilename = "chessboard_images/drawn_" +
fs::path(filename).filename().string();
        imwrite(outputFilename, frame);
    }
}

//Update na ImageFiles list

ImagesPage::uploadUndist(referenceLabel, refimageSelector,
std::ref(refs), "chessboard_images/");
//destroyAllWindows();

if (imagePoints.empty()) {
    cerr << "\nError: No valid images for calibration." << endl;
    QMessageBox::warning(this, "Camera Info upload failed", "Error:
No valid images for calibration.\n\n Please be certain that you selected
the right amount of columns and rows, and that you uploaded valid
images");
    progressBar->setValue(0);
    return;
}

```

```

    }

    //Mat cameraMatrix, distCoeffs;
    vector<Mat> rvecs, tvecs;
    progressBar->setValue(90);

    //if (flagpar == 1) {
    cv::calibrateCamera(objectPoints, imagePoints, gray.size(),
cameraMatrix, distCoeffs, rvecs, tvecs);

    //}
    //else {
    //    cv::calibrateCamera(objectPoints, imagePoints, gray.size(),
cameraMatrix, distCoeffs, rvecs, tvecs);
    //}

    ofstream file("calibration_results.txt");
    if (file.is_open()) {
        file << "Camera Matrix:\n" << cameraMatrix << "\n";
        file << "Distortion Coefficients:\n" << distCoeffs << "\n";
        file.close();
        cout << "\nCalibration results saved to
'calibration_results.txt'." << endl;
    }
    else {
        cerr << "\nError: Unable to save calibration data." << endl;
        return;
    }

    string coeffs;
    QFile camcoeffs("calibration_results.txt");
    if (camcoeffs.open(QIODevice::ReadOnly | QIODevice::Text)) {
        coeffs = camcoeffs.readAll();
        camcoeffs.close();
        QDate date(QDate::currentDate());
        info->setPlainText(QString::fromStdString(coeffs));
    }
    else {
        info->setPlainText("\nError: Unable to open the file.");
    }

    // Create directory for undistorted images
    if (!fs::exists("undistorted_images")) {
        fs::create_directory("undistorted_images");
    }
    fs::remove_all("undistorted_images/");
    if (!fs::exists("undistorted_images/")) {
        fs::create_directory("undistorted_images/");
    }

    imageFiles.clear();
    for (const auto& entry : fs::directory_iterator("images")) {
        if (entry.path().extension() == ".jpg" ||
entry.path().extension() == ".png") {
            imageFiles.push_back(entry.path().string());
            cout << "\n";
            cout << entry.path().string() << endl;
        }
    }
}

```

```

// Save and show undist images
for (const auto& filename : imageFiles) {
    Mat original = imread(filename);
    if (original.empty()) {
        cerr << "\nWarning: Could not read " << filename << endl;
        continue;
    }

    Mat undistorted;
    undistort(original, undistorted, cameraMatrix, distCoeffs);

    string outputFilename = "undistorted_images/undistorted_" +
fs::path(filename).filename().string();
    imwrite(outputFilename, undistorted);

    //Upload para references
}
ImagesPage::uploadUndist(referenceLabel, refimageSelector,
std::ref(refs), "undistorted_images/");
progressBar->setValue(0);
flagpar = 0;
}

void ImagesPage::closedb() {
    if (db) {
        sqlite3_close(db);
        db = nullptr;
    }
}

CameraPage::CameraPage(QWidget* parent)
: QWizardPage(parent)
{
    setTitle("Live Calibration");
    zoom = new QSlider;
    zoom->setOrientation(Qt::Horizontal);
    zoom->setFixedWidth(500);
    zoom->setMaximum(500);
    zoom->setMinimum(0);
    zoom->setSliderPosition(100);
    zoompx = new QLabel;
    framedisplay = new QGraphicsView;
    framedisplay->setFixedHeight(300);
    framedisplay->setFixedWidth(550);

    videoWidget = new QVideoWidget;
    videoWidget->setFixedHeight(280);
    videoWidget->setFixedWidth(500);
    combocamaras = new QComboBox;
    paracamara = new QPushButton;
    paracamara->setText("Stop Camera");
    paracamara->setFixedHeight(24);
    paracamara->setFixedWidth(90);
    saveframe = new QPushButton;
    saveframe->setText("Save Frame");
    saveframe->setFixedHeight(24);
    saveframe->setFixedWidth(90);
    mediacapture = new QMediaCaptureSession();

```

```

currentcam = new QCamera();
qvs = new QVideoSink();
scene = new QGraphicsScene();
livecalbutton = new QPushButton();
livecalbutton->setText("OpenCV Live Calibration");
undistbutton = new QPushButton();
undistbutton->setText("Open camera with distortion removal effect");

connect(combocamaras, &QComboBox::currentIndexChanged, this,
&CameraPage::selectcamera);
connect(paracamara, &QPushButton::clicked, this,
&CameraPage::on_paracamara_clicked);
connect(zoom, &QSlider::valueChanged, this,
&CameraPage::on_zoom_valueChanged);
connect(livecalbutton, &QPushButton::clicked, this,
&CameraPage::livecalibration);
connect(undistbutton, &QPushButton::clicked, this,
&CameraPage::undistcamera);
connect(saveframe, &QPushButton::clicked, this,
&CameraPage::on_saveframe_clicked);

getCameras();
QGridLayout* layout = new QGridLayout;
layout->addWidget(videoWidget, 0, 0);
layout->addWidget(combocamaras, 1, 0);
layout->addWidget(paracamara, 2, 0);
layout->addWidget(framedisplay, 0, 1);
layout->addWidget(zoom, 1, 1);
layout->addWidget(zoompx, 2, 1);
layout->addWidget(saveframe, 2, 1, Qt::AlignRight);
layout->addWidget(livecalbutton, 3, 0);
layout->addWidget(undistbutton, 3, 1);
setLayout(layout);
}
void CameraPage::getCameras()
{
    combocamaras->addItem("<None>");
    //coloca na lista "cameras" todos os dispositivos a captar video
    cameras = QMediaDevices::videoInputs();
    //adiciona à comboBox essa lista
    for (const QCameraDevice& camera : cameras)
    {
        combocamaras->addItem(camera.description());
    }
}
void CameraPage::selectcamera(int index)
{
    //contexto: esta função é ativada sempre que o index da ui.comboBox é
    mudado
    //da reset na camera que esta em uso
    if (currentcam->isActive())
    {
        currentcam->stop();
    }
    //a partir da lista de camaras procura a que tem a mesma descrição da
    selecionada
    for (const QCameraDevice& camera : cameras)
    {
        if (camera.description() == combocamaras->currentText())
        {

```

```

        //passa a camara em uso para a que corresponde ao indice da
comboBox
        currentcam->setCameraDevice(camera);
        //mediacapturesession para poder transmitir o video para o
widget da classe QVideoWidget
        mediacapture->setCamera(currentcam);
        mediacapture->setVideoOutput(videoWidget);
        //cria um video sink associado a esta mediacapture para
extrair frames (ver a função on_paracamara_clicked)
        qvs = mediacapture->videoSink();
        currentcam->start();
        break;
    }
}
}
void CameraPage::on_zoom_valueChanged()
{
    //troca o texto que representa a posicao do slider "zoom"
    zoompx->setText("Zoom: " + QString::number(zoom->value()) + "%");
    // extrai o valor do slider para uma variavel e calcula a escala
    qreal scaleFactor = static_cast<qreal>(zoom->value()) / 100.0;

    framedisplay->resetTransform();
    framedisplay->scale(scaleFactor, scaleFactor);
}
void CameraPage::on_paracamara_clicked()
{
    //contexto: esta função funciona quando o botao "para camara" é
clicado, ui.framedisplay é um objeto do tipo QGraphicsView
    if (!qvs)
        return;
    // usa o video sink associado em selectcamera para tirar um frame
    QVideoFrame frame = qvs->videoFrame();
    if (!frame.isValid())
        return;
    // converte o frame em imagem e passa de imagem para pixmap
    QImage img = frame.toImage();
    img.save("imageslive/frame.png", "png"); // Save the frame as an
image file

    Mat original = imread("imageslive/frame.png");

    Mat undistorted;
    undistort(original, undistorted, cameraMatrix, distCoeffs);

    img=MatToQImage(undistorted); // Save the undistorted frame

    QPixmap pixmap = QPixmap::fromImage(img);
    // usa uma QGraphicsScene para colocar no graphicsview
    scene->clear();
    scene->addPixmap(pixmap);
    // muda a scene e mostra no graphicsview
    framedisplay->setScene(scene);
    framedisplay->show();
}
void CameraPage::on_saveframe_clicked()
{
    if (!framedisplay || !scene)
        return;

    // Get the zoom scale from the slider

```



```

qreal scaleFactor = static_cast<qreal>(zoom->value()) / 100.0;

// Guarda a parte visível do framedisplay com o zoom aplicado
QSize viewSize = framedisplay->viewport()->size();
QSize scaledSize = viewSize * scaleFactor;

// Renderiza o que está no framedisplay para um QPixmap
QPixmap pixmap(scaledSize);
QPainter painter(&pixmap);
framedisplay->render(&painter);
painter.end();

// Save the final image
QString baseName = "imageslive/saved_frame.jpg";
int counter = 0;
QString filename;
do {
    filename = baseName + QString::number(counter++) + ".jpg";
} while (QFile::exists(filename));

if (pixmap.save(filename)) {
    QMessageBox::information(this, "Frame Saved", "Image saved
successfully to:\n" + filename);
}
else {
    QMessageBox::warning(this, "Save Failed", "Failed to save the
image.");
}
}
float CameraPage::livecalibration() {

    flaglivecam = 1;
    VideoCapture cap(0);
    if (!cap.isOpened()) {
        cerr << "Error: Unable to open the camera." << endl;
    }

    vector<Point3f> objp;
    for (int i = 0; i < boardHeight; i++) {
        for (int j = 0; j < boardWidth; j++) {
            objp.push_back(Point3f(j * squareSize, i * squareSize, 0));
        }
    }

    Mat frame, gray;
    int numFrames = 0;
    while (numFrames < 15) { // Capture 15 valid images
        cap >> frame;
        if (frame.empty()) {
            cerr << "Error: Frame capture failed." << endl;
            break;
        }

        cvtColor(frame, gray, COLOR_BGR2GRAY);
        vector<Point2f> corners;
        bool found = findChessboardCorners(gray, boardSize, corners,
            CALIB_CB_ADAPTIVE_THRESH | CALIB_CB_FAST_CHECK |
            CALIB_CB_NORMALIZE_IMAGE);

        if (found) {
            cornerSubPix(gray, corners, Size(11, 11), Size(-1, -1),

```

```

        TermCriteria(TermCriteria::EPS + TermCriteria::COUNT, 30,
0.001));

        drawChessboardCorners(frame, boardSize, corners, found);
        imagePoints.push_back(corners);
        objectPoints.push_back(objp);

        cout << "Captured frame " << numFrames + 1 << endl;
        numFrames++;
    }

    imshow("Camera Calibration", frame);
    char key = (char)waitKey(2000);
    if (key == 27) { // Press 'ESC' to exit
        break;
    }
}

cap.release();
cv::waitKey(1);
destroyAllWindows();

// Camera calibration
vector<Mat> rvecs, tvecs;

cv::calibrateCamera(objectPoints, imagePoints, gray.size(),
cameraMatrix2, distCoeffs2, rvecs, tvecs);

cout << "Camera Matrix: \n" << cameraMatrix2 << endl;
cout << "Distortion Coefficients: \n" << distCoeffs2 << endl;

// Save calibration data
ofstream file("calibration_resultslive.txt");
if (file.is_open()) {
    file << "Camera Matrix:\n" << cameraMatrix2 << "\n";
    file << "Distortion Coefficients:\n" << distCoeffs2 << "\n";
    file.close();
}

}

void CameraPage::undistcamera()
{
    VideoCapture cap(0);
    if (!cap.isOpened()) {
        QMessageBox::warning(
            this,
            "Warning",
            "Error: Unable to open the camera for distortion display."
        );
        return;
    }

    // Grab one frame to get the size
    Mat frame;
    cap >> frame;
    if (frame.empty()) {
        cerr << "Error: Frame capture failed." << endl;
        cap.release();
        return;
    }
}

```

```

    // Check calibration data
    if (cameraMatrix2.empty() || distCoeffs2.empty()) {
        QMessageBox::warning(this, "Warning", "Calibration data not loaded.
Please calibrate the camera first.");
        cap.release();
        return;
    }

    // (Optional) Get optimal new camera matrix
    Mat newCameraMatrix = getOptimalNewCameraMatrix(cameraMatrix2,
distCoeffs2, frame.size(), 1);

    // Precompute undistortion maps
    Mat map1, map2;
    initUndistortRectifyMap(cameraMatrix2, distCoeffs2, Mat(),
newCameraMatrix, frame.size(), CV_32FC1, map1, map2);

    while (true) {
        cap >> frame;
        if (frame.empty()) {
            cerr << "Error: Frame capture failed." << endl;
            break;
        }

        Mat undistortedFrame;
        remap(frame, undistortedFrame, map1, map2, INTER_LINEAR,
BORDER_CONSTANT);
        //cv::undistort(frame, undistortedFrame, cameraMatrix2,
distCoeffs2);

        imshow("Undistorted Camera View", undistortedFrame);

        char key = (char)waitKey(30);
        if (key == 27) { // Press 'ESC' to exit
            break;
        }
    }

    cap.release();
    cv::waitKey(1);
    destroyAllWindows();
}

ConclusionPage::ConclusionPage(QWidget* parent)
    : QWizardPage(parent)
{
    setTitle(tr("Conclusion"));
    setPixmap(QWizard::WatermarkPixmap,
QPixmap(":/images/watermark2.png"));

    label = new QLabel;
    label->setWordWrap(true);

    QVBoxLayout* layout = new QVBoxLayout;
    layout->addWidget(label);
    setLayout(layout);
}

void ConclusionPage::initializePage()
{

```

```

setTitle("Calibration Completed");
QString finishText = wizard()->buttonText(QWizard::FinishButton);
finishText.remove('&');

    setSubTitle("The calibration process (images, and calibration results
such as distortion coefficients and intrinsic parameters) are saved in
the source folder\n\n"
    "Click the finish button to end this wizard\n\n"
    );
}

```